



## Assessment Cover Sheet

|                      |                                  |             |           |
|----------------------|----------------------------------|-------------|-----------|
| Assessment           | Project (Individual)             |             |           |
| Assessment           | Uncontrolled                     | Individual  | Must-pass |
| Due Date             | 27-05-2026                       | Course Code | IT9201    |
| Course Title         | Machine Learning and Data mining |             |           |
| Internal Moderator's | Dr. David Gulua                  |             |           |
| External Examiner's  |                                  |             |           |

**Instructions:**

1. This cover sheet must be completed (section in red below) and attached to your assessment before submission in hard copy/soft copy.
2. The time allowed for this assessment is until 27-05-2026 -11:59 PM.
3. This assessment carries 100 marks assessing CLIO1, CLIO2, and CLIO3.
4. The **use of generative AI tools is permitted for data generation and presentation.**
5. References consulted (if any) must be properly acknowledged and cited.

|              |                                              |                |  |
|--------------|----------------------------------------------|----------------|--|
| Learner ID   | 202508958                                    | Date Submitted |  |
| Learner Name | Fatema Husain Hasan                          |                |  |
| Programme    | ICT9010 - MSc in Artificial Intelligence     |                |  |
| Programme    | Master of Science in Artificial Intelligence |                |  |
| Lecturer's   | Dr. Shomona Gracia Jacob                     |                |  |

**By submitting this assessment for marking, I affirm that this assessment is my own work.**

Learner

Do not write beyond this line. For assessor use only.

|                 |  |               |  |
|-----------------|--|---------------|--|
| Assessor's Name |  |               |  |
| Marking Date    |  | Maks Obtained |  |

Comments:

# Table of Contents

|                                                                                                  |    |
|--------------------------------------------------------------------------------------------------|----|
| Table of Tables .....                                                                            | 3  |
| Table of Figures .....                                                                           | 3  |
| Task 1: Problem and Objectives Background of the Study .....                                     | 4  |
| 1.1 Problem Definition.....                                                                      | 4  |
| 1.2 Literature Review.....                                                                       | 4  |
| 1.3 Objectives.....                                                                              | 5  |
| 1.4 Experimental Setup.....                                                                      | 5  |
| Task 2: Proposed ML framework with Data Acquisition and Processing .....                         | 6  |
| 2.1 Dataset Description .....                                                                    | 6  |
| 2.2 Justification for Dataset Choice.....                                                        | 7  |
| 2.3 Data Preprocessing .....                                                                     | 8  |
| Task 3: Choice of feature selection, learning methods and evaluation strategies and metrics..... | 11 |
| 3.1 Feature Selection - Chi-Square.....                                                          | 11 |
| 3.2 Supervised Learning .....                                                                    | 12 |
| 3.2.1 K-Nearest Neighbor (KNN) .....                                                             | 12 |
| 3.2.2 Decision Tree .....                                                                        | 13 |
| 3.3 Unsupervised Learning - K-Means Clustering .....                                             | 15 |
| 3.4 Association Rule Mining - Apriori .....                                                      | 19 |
| 3.5 Evaluation Metrics .....                                                                     | 20 |
| Task 4: Hyper-parameter optimization – Summarization of results and inference .....              | 21 |
| 4.1 KNN Hyperparameter Tuning .....                                                              | 21 |
| 4.2 Decision Tree Hyperparameter Tuning.....                                                     | 22 |
| 4.3 K-Means Hyperparameter Tuning.....                                                           | 24 |
| 4.4 MindSpore MLP Hyperparameter Tuning .....                                                    | 25 |
| 4.5 Summary of All Models .....                                                                  | 26 |
| Task 5: Discussions and Future Extensions to the Work .....                                      | 28 |
| 5.1 Critical Analysis of Results .....                                                           | 28 |
| 5.2 Comparison to Previous Work .....                                                            | 29 |
| 5.3 Pros and Cons of Each Model.....                                                             | 29 |
| 5.4 How the Framework Addresses the Objectives.....                                              | 30 |
| 5.5 Limitations and Future Work.....                                                             | 30 |
| 5.5.1 Limitations.....                                                                           | 30 |
| 5.5.2 Future Work .....                                                                          | 31 |
| Use of GenAI.....                                                                                | 31 |
| References.....                                                                                  | 32 |

## Table of Tables

|                                                               |    |
|---------------------------------------------------------------|----|
| Table 1 - Experimental Setup Table .....                      | 5  |
| Table 2 - Dataset Description.....                            | 6  |
| Table 3 - KNN Default Performance (k=5).....                  | 12 |
| Table 4 - Decision Tree Default Performance (depth=None)..... | 13 |
| Table 5 - K-Means Clustering Results (k=2) .....              | 17 |
| Table 6 - Top Association Rules from Apriori Mining.....      | 19 |
| Table 7 - KNN Hyperparameter Tuning Results.....              | 21 |
| Table 8 - Decision Tree Hyperparameter Tuning Results .....   | 22 |
| Table 9 - K-Means Elbow Method and Silhouette Scores .....    | 24 |
| Table 10 - MindSpore MLP Hyperparameter Tuning Results .....  | 25 |
| Table 11 - Consolidated Model Performance Summary .....       | 26 |
| Table 12 - Comparison with Related Work .....                 | 29 |
| Table 13 - Model Pros and Cons .....                          | 29 |

## Table of Figures

|                                                                                             |    |
|---------------------------------------------------------------------------------------------|----|
| Figure 1 - Class Distribution Bar Chart .....                                               | 6  |
| Figure 2 - Class Distribution Pie Chart .....                                               | 7  |
| Figure 3 - Convert the dataset from ARFF to CSV .....                                       | 8  |
| Figure 4 - Exploring the dataset .....                                                      | 8  |
| Figure 5 - Correlation Heatmap .....                                                        | 9  |
| Figure 6 - Engineered feature distributions .....                                           | 9  |
| Figure 7 - Train and Test Split.....                                                        | 10 |
| Figure 8 - Dataset Overview and Set a Target .....                                          | 10 |
| Figure 9 - Chi-Square feature importance bar chart .....                                    | 12 |
| Figure 10 - KNN Confusion Matrix .....                                                      | 13 |
| Figure 11 - Decision Tree Confusion Matrix .....                                            | 14 |
| Figure 12 - Decision Tree Structure (first 3 levels).....                                   | 14 |
| Figure 13 - Orange Test and Score results showing KNN performance.....                      | 15 |
| Figure 14 - Elbow Method plot.....                                                          | 16 |
| Figure 15 - Silhouette Score vs k plot.....                                                 | 16 |
| Figure 16 - PCA Scatter Plot (clusters vs true labels) .....                                | 17 |
| Figure 17 - Orange K-Means workflow.....                                                    | 18 |
| Figure 18 - Orange Scatter Plot (SSLfinal_State vs URL_of_Anchor coloured by cluster) ..... | 18 |
| Figure 19 - Orange Silhouette Plot .....                                                    | 19 |
| Figure 20 - Association Rules scatter plot (Support vs Confidence) .....                    | 20 |
| Figure 21- KNN Performance vs k (line chart) .....                                          | 21 |
| Figure 22 - KNN Tuned Confusion Matrix (k=3) .....                                          | 22 |
| Figure 23 - Decision Tree Performance vs max_depth (line chart) .....                       | 23 |
| Figure 24 - Decision Tree Tuned Confusion Matrix (depth=None) .....                         | 23 |
| Figure 25 - Decision Tree Feature Importance chart .....                                    | 24 |
| Figure 26 - MindSpore Hyperparameter Accuracy bar chart.....                                | 25 |
| Figure 27 - MindSpore MLP Confusion Matrix.....                                             | 26 |
| Figure 28 - All Models Comparison bar chart (Accuracy, F1, Recall) .....                    | 27 |

# Task 1: Problem and Objectives Background of the Study

## 1.1 Problem Definition

Phishing is a fraudulent process in the online world whereby malicious websites are designed to closely resemble legitimate information services, such as social media, banking, and online shopping, in order to trick users into using phishing services to disclose financial information, personal information, and necessary login credentials. Phishing is really the most common type of cybercrime, with damages estimated by the FBI Internet Crime Report (2023) to be over \$12.5 billion globally. Every month, more than 1.5 million phishing websites are formed, and compiling a list of them by hand is almost impossible. Machine learning may be used to create scalable, automated rules of statistical patterns between spam and legitimate websites based on the features of the URL (Tang & Mahmoud, 2021). Based on the UCI Phishing Websites Dataset (Mohammad & McCluskey, 2012), this study develops and assesses phishing website classification models that can identify phishing websites in real-time using supervised and unsupervised machine learning approaches.

## 1.2 Literature Review

Over the past ten years, there has been a lot of academic interest in machine learning-based phishing detection. By generating 30 URL-based features that are encoded as ordinal numbers, Mohammad and McCluskey (2012) demonstrated that rule-based classifiers may obtain dependable detection performance using a basic benchmark dataset. Since then, several additional research validated this dataset, making it a common benchmark.

Tang and Mahmoud (2021) conducted a comprehensive examination of machine learning algorithms for phishing detection and concluded that, although interpretability remains a critical deployment gap, ensemble approaches and feature-engineered datasets consistently outperform single classifiers. Hannousse and Yahiouche (2021) investigated several classifiers on different phishing datasets in their paper, confirming the important impact that dataset variables like feature encoding and class balancing have on classifier performance.

Recently, Chawla (2022) achieved the highest accuracy of 97.73% on the UCI dataset using KNN, Decision Tree, Logistic Regression, and Max Vote ensemble of Random Forest, Decision Tree, and ANN. Additionally, using the same UCI benchmark dataset, Choudhary et al. (2023) showed the improved generalization performance of k-fold cross-validation with feature selection, with the Random Forest model achieving 97.87% accuracy on the UCI dataset. Eight methods were compared using the UCI and Mendeley phishing datasets by Haq et al. (2024). They discovered that while deep learning techniques are less interpretable, they are more accurate. Nevertheless, these studies have significant flaws that must be fixed, such as the lack of unsupervised learning analysis and the failure to use the no-code tool (Chawla, 2022; Hannousse & Yahiouche, 2021; Choudhary et al., 2023). In this research, K-Means is merged with supervised classifiers, and the entire operation is replicated in Orange to address both issues.

## 1.3 Objectives

1. Implement and evaluate the KNN and Decision Tree classifiers for the detection of phishing websites.
2. Apply K-Means clustering, which is used to find unsupervised phishing patterns.
3. Apply Chi-Square for feature selection to determine the most discriminative URL features.
4. Tune hyperparameters for all models and compare the default performance to optimized performance.
5. Transfer to MindSpore to further show the transferability of the framework.

## 1.4 Experimental Setup

This project was implemented using both code-based and no-code tools as required by the project brief. The table below shows the full experimental configuration used across all tasks.

| Component            | Details                                                                              |
|----------------------|--------------------------------------------------------------------------------------|
| Programming Language | Python 3.x                                                                           |
| Code-based Tool      | Jupyter Notebook (VS Code)                                                           |
| No-code Tool         | Orange Data Mining                                                                   |
| Migration Framework  | MindSpore 2.8.0 - CPU platform                                                       |
| Libraries            | pandas, numpy, scikit-learn, matplotlib, seaborn, mlxtend, mindspore                 |
| Evaluation Strategy  | 80/20 stratified train/test split (scikit-learn)<br>5-fold cross validation (Orange) |
| Hardware             | CPU-based execution                                                                  |
| Dataset Source       | UCI Machine Learning Repository (Mohammad & McCluskey, 2012)                         |

Table 1 - Experimental Setup Table

# Task 2: Proposed ML framework with Data Acquisition and Processing

## 2.1 Dataset Description

The data set chosen for this project is Phishing Websites Dataset from the UCI Machine Learning Repository (UCI ML) collected by Mohammad and McCluskey in 2012. With its 2012 origin, the dataset is still relevant and widely used as a phishing detection benchmark for research (Chawla, 2022; Hannousse & Yahiouche, 2021; Haq et al., 2024). This dataset contains 11,055 rows representing instances and 31 columns representing features, out of which 30 are features as input and 1 is the target variable which is (Result). The 30 features are pre-encoded as ordinal integers that encode the URL-based, domain-based and third-party service-based characteristics of websites.

| Attribute          | Details                                                                              |
|--------------------|--------------------------------------------------------------------------------------|
| Total Instances    | 11,055                                                                               |
| Feature Columns    | 30                                                                                   |
| Target Column      | Result (+1 = Legitimate, -1 = Phishing)                                              |
| Feature Encoding   | Ordinal integers: -1 (phishing indicator), 0 (suspicious), +1 (legitimate indicator) |
| Missing Values     | None                                                                                 |
| Class Distribution | 6,157 Legitimate (55.7%) / 4,898 Phishing (44.3%)                                    |
| Imbalance Ratio    | 1.26:1 - near-balanced                                                               |

Table 2 - Dataset Description

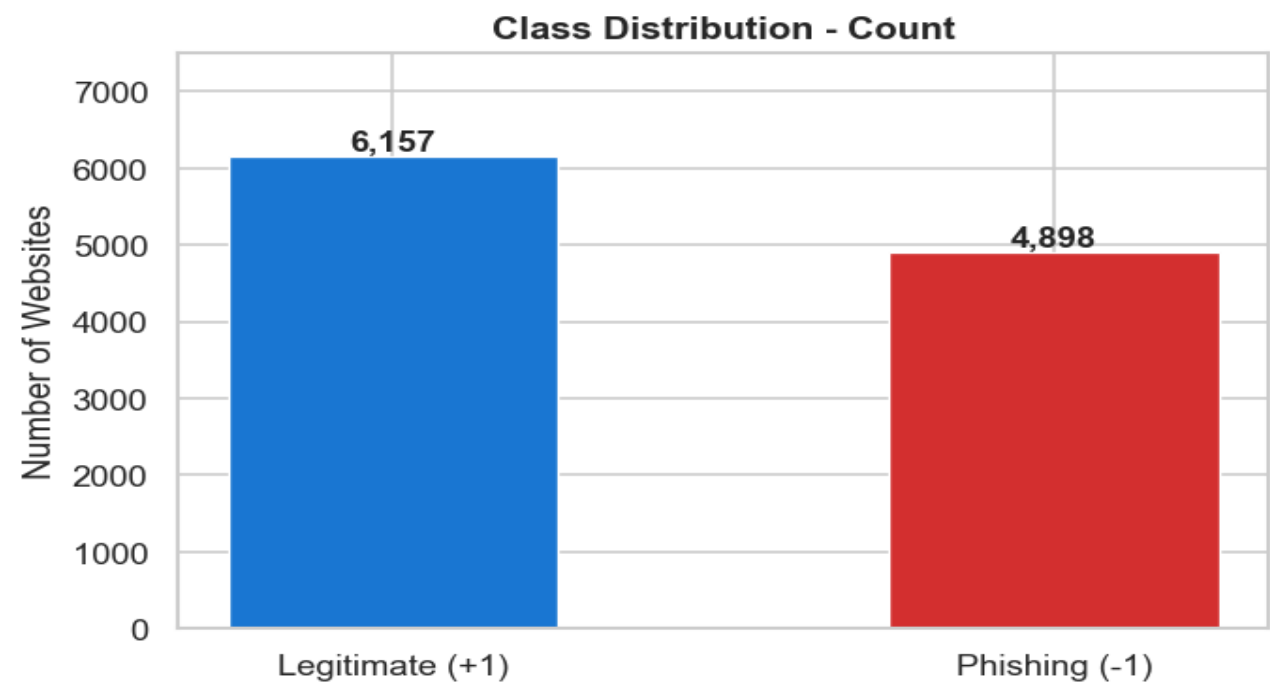


Figure 1 - Class Distribution Bar Chart

## Class Distribution - Percentage

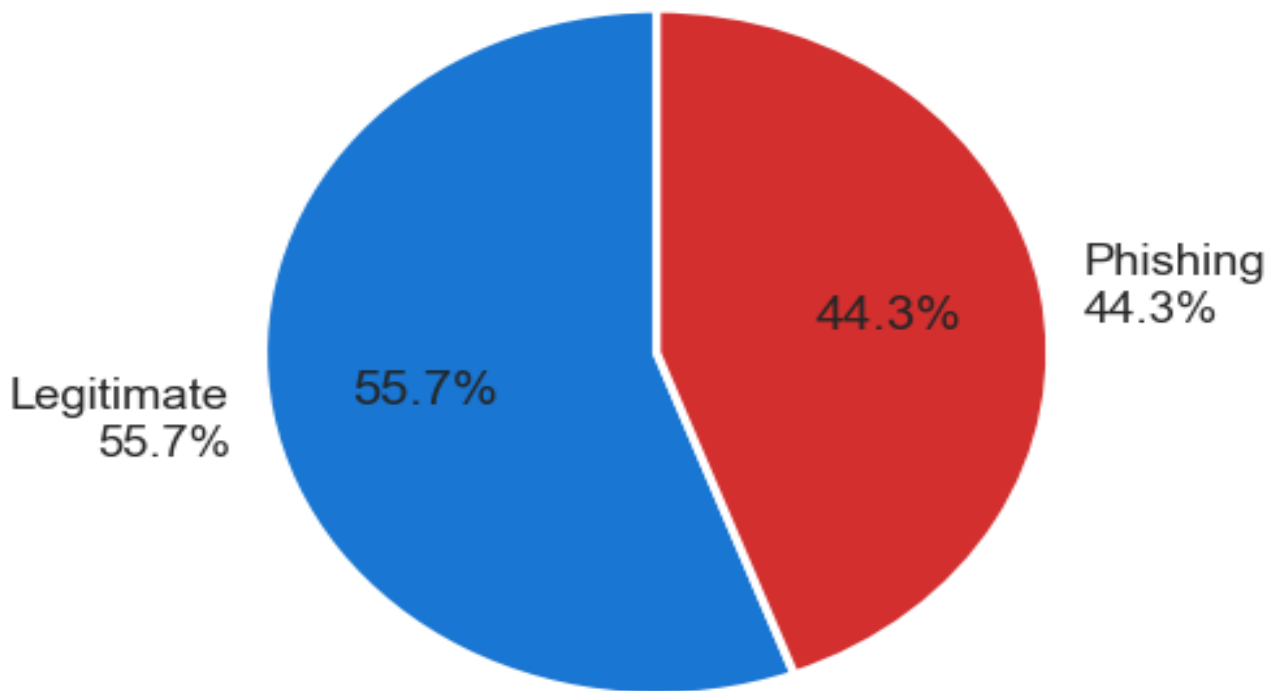


Figure 2 - Class Distribution Pie Chart

## 2.2 Justification for Dataset Choice

The decision to select the UCI Phishing Websites Dataset (Mohammad & McCluskey, 2012) was made due to its relevance in the professional field, technical usability, statistical characteristics, and credibility in the research community. For a cybersecurity employee like me, phishing detection is an operational concern. One of the most common threats in real-world IT systems is phishing, and it is feasible to create an automated machine learning-based detection framework in a professional cybersecurity setting. As a result, this initiative is firmly situated at the nexus of practical industry relevance and academic research, with the dataset selection being driven by industry as well as academic norms.

Without requiring complicated preparation of the text or feature extraction procedure, the dataset is organized in a tabular style and contains 30 pre-encoded numeric features that immediately translate to all three machine learning techniques used in this project: KNN, Decision Tree, and K-Means. Compared to raw URL datasets, which required a significant natural language processing effort, this dataset is more suited to concentrate the study on model creation, assessment, and interpretation (Hannousse & Yahiouche, 2021). Moreover, this dataset has been used in all peer-reviewed papers published between 2021 and 2024 (Chawla, 2022; Choudhary et al., 2023; Hannousse & Yahiouche, 2021), making it a strong and reliable basis for comparing the results of this project with known results. This academic validation extends the continued suitability and relevance of this dataset as a research benchmark, given its origins in 2012.

## 2.3 Data Preprocessing

This pre-processing pipeline is run in Jupyter Notebook and then reproduced in Orange:

**Step 1 ARFF to CSV conversion:** The data was downloaded from UCI in ARFF data format and converted to CSV using a custom written function in Python called `parser_arff[]` that parses the ARFF data file to capture the column names under the attribute section and the data under the data section.

```
def arff_to_csv(arff_path, csv_path):
    """Convert UCI ARFF file to CSV - reads @attribute for columns, @data for rows."""
    columns = []
    data_section = False
    rows = []

    with open(arff_path, 'r', encoding='utf-8') as f:
        for line in f:
            line = line.strip()
            if not line or line.startswith('%'):
                continue
            if line.lower().startswith('@attribute'):
                parts = line.split()
                col_name = parts[1].strip('"\'')
                columns.append(col_name)
            elif line.lower().startswith('@data'):
                data_section = True
            elif data_section and line:
                rows.append(line.split(','))

    df = pd.DataFrame(rows, columns=columns)
    for col in df.columns:
        df[col] = pd.to_numeric(df[col], errors='coerce')
    df.to_csv(csv_path, index=False)

    print(f"Conversion complete!")
    print(f" Rows   : {df.shape[0]:,}")
    print(f" Columns : {df.shape[1]:,}")
    print(f" Saved   : {csv_path}")
    return df

df = arff_to_csv(r'C:\Users\Fatim\OneDrive\Desktop\Data Mining Project\phishing+websites\Training Dataset.arff', 'phishing_dataset.csv')
```

[3] ✓ 0.2s Python

... Conversion complete!  
Rows : 11,055  
Columns : 31  
Saved : phishing\_dataset.csv

Figure 3 - Convert the dataset from ARFF to CSV

**Step 2 Exploratory data analysis (EDA):** A view was performed on the shape of the dataset, data types, missing data and distribution of classes. There were no missing values. All features were confirmed as being integers in the range of {-1, 0, 1}.

```
# Load the converted CSV
df = pd.read_csv('phishing_dataset.csv')

any_missing = df.isnull().values.any()

print("=" * 55)
print(" DATASET OVERVIEW")
print("=" * 55)
print(f" Rows       : {df.shape[0]:,}")
print(f" Columns    : {df.shape[1]:,}")
print(f" Feature cols : {df.shape[1] - 1}")
print(f" Target column : Result")
print(f" Missing values : {any_missing}")
print(f" Feature values : {sorted(df.drop('Result', axis=1).stack().unique())}")
print()
print("Feature names:")
for i, col in enumerate(df.columns, 1):
    print(f" {i:2d}. {col}")
```

[4] ✓ 0.0s Python

... DATASET OVERVIEW

Rows : 11,055  
Columns : 31  
Feature cols : 30  
Target column : Result  
Missing values : False  
Feature values : [-1, 0, 1]

Feature names:  
1. having\_IP\_Address  
2. URL\_Length  
3. Shortning\_Service

Figure 4 - Exploring the dataset



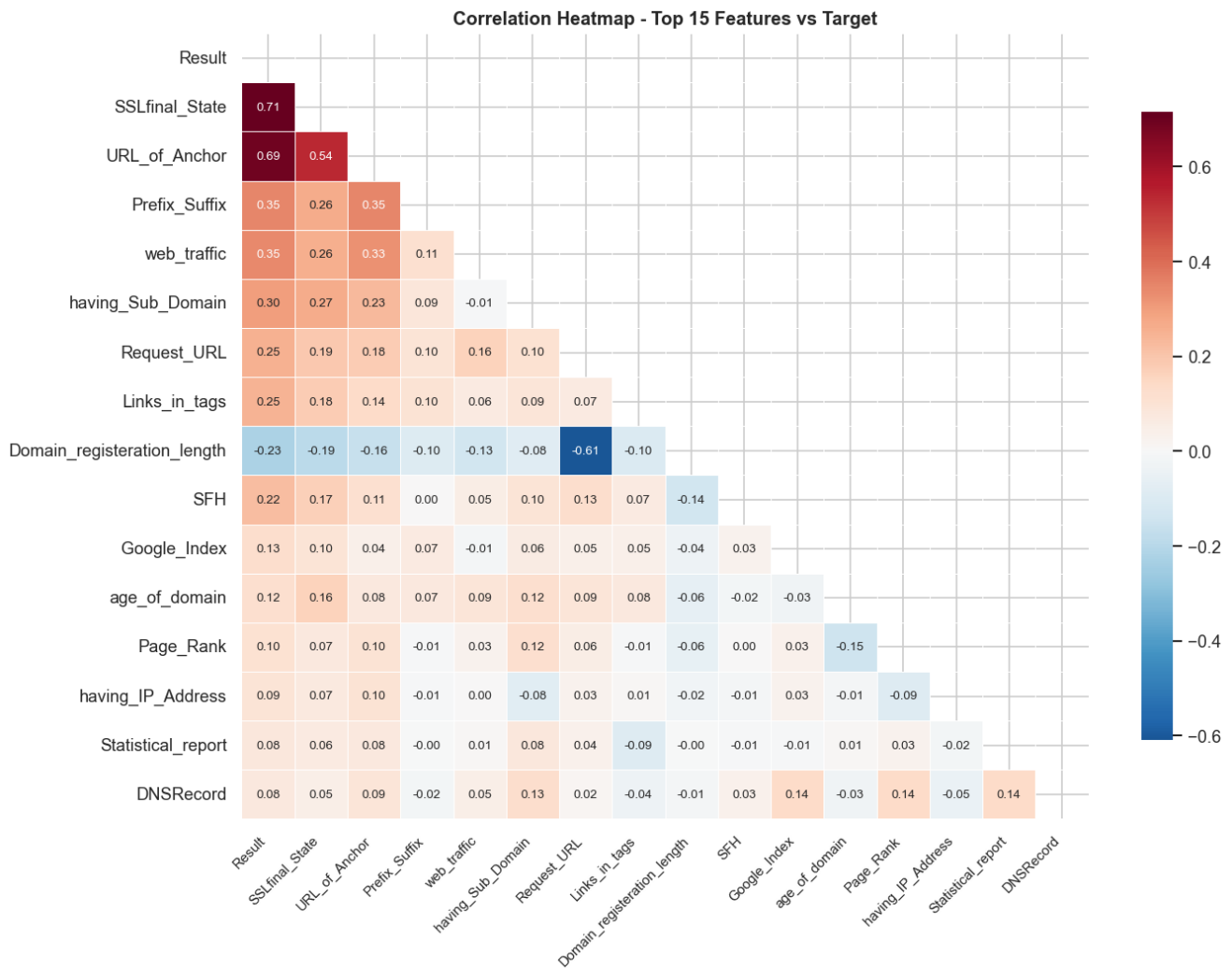


Figure 5 - Correlation Heatmap

**Step 3 Feature Engineering:** Using domain knowledge, two new features were developed Domain\_Trust\_Score (count of domain credibility indicators) and URL\_Risk\_Score (count of URL-based phishing indicators). Figures 6 and 5 demonstrate the distinct distributional disparities between groups for both traits.

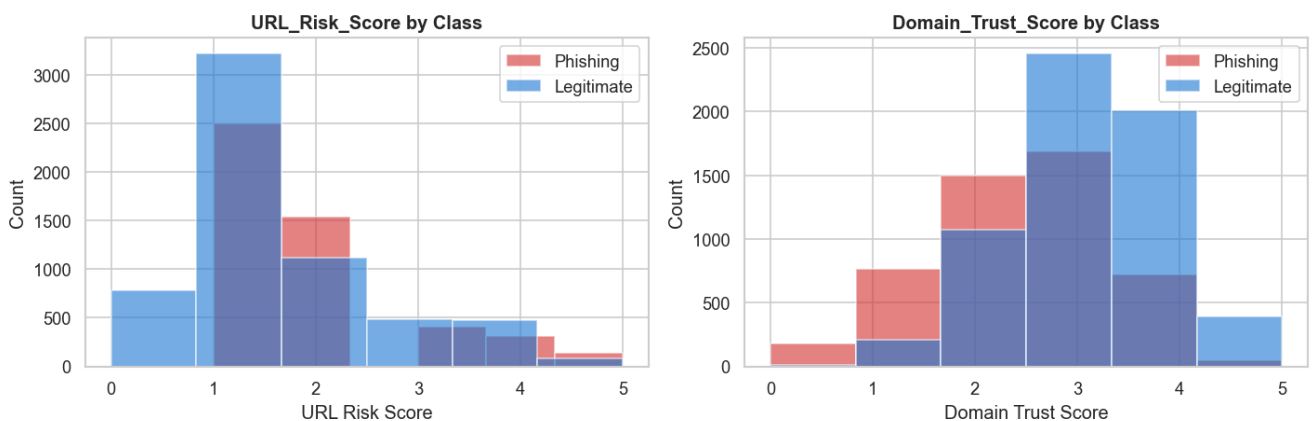


Figure 6 - Engineered feature distributions

**Step 4 Train and Test Split:** Stratified splitting with random state=42 was used to ensure class balance, and 80% of the data was used for training (8,844 instances), with the remaining 20% set aside for testing (2,211 instances).

```

# 2. Train/test split 80/20 - stratified
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=RANDOM_STATE, stratify=y
)

print(f"\nTrain set : {X_train.shape[0]:,} rows ({X_train.shape[0]/len(X)*100:.0f}%)")
print(f"Test set : {X_test.shape[0]:,} rows ({X_test.shape[0]/len(X)*100:.0f}%)")
print(f"Stratified: Yes - class balance preserved")

```

[18] ✓ 0.0s  
 ...  
 Train set : 8,844 rows (80%)  
 Test set : 2,211 rows (20%)  
 Stratified: Yes - class balance preserved

Figure 7 - Train and Test Split

**Step 5 Standard Scaler:** The feature normalization was applied only to KNN and K-Means (both are distance-based algorithms) to standardize the features. As Decision Tree is rules-based and scale independent, it was trained using unscaled data.

**Orange implementation:** A pipeline was implemented in Orange with the same widgets using File widget (with Result set as the target variable, and categorical type); Preprocess widget (normalization); Select Columns widget (feature management).

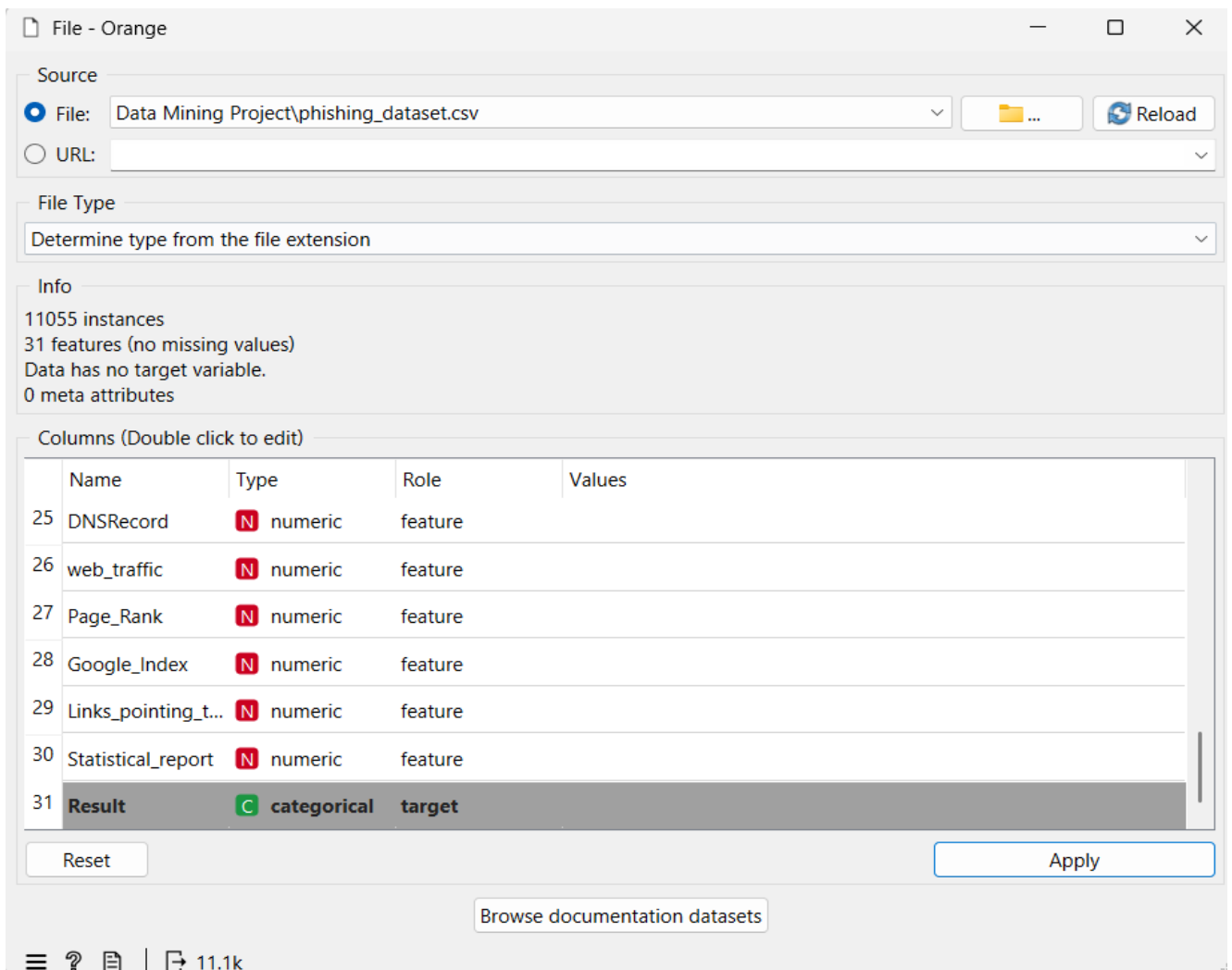


Figure 8 - Dataset Overview and Set a Target

## Task 3: Choice of feature selection, learning methods and evaluation strategies and metrics

### 3.1 Feature Selection - Chi-Square

Feature selection was carried out by Chi-Square which is a filter method. Based on the following hypothesis structure, Chi-Square measures the statistical association between each attribute and the desired value:

$H_0$ : The feature is INDEPENDENT of whether a website is phishing or legitimate

$H_1$ : The feature is statistically related to the target class.

The result is significant and  $H_0$  is rejected if the p value is less than 0.05. Fifteen features were determined to be statistically significant after the test was run on all thirty features. The SSL certificate status (3,753.8;  $p = 0.000$ ) has the greatest Chi-Square, indicating that it is the most significant discriminative factor. A Pearson correlation score of  $r = 0.71$  with the target variable and a Decision Tree feature importance score of 0.622, which was derived from three different approaches and demonstrated the significance of the target variable, further supported this outcome (Hannousse & Yahiouche, 2021). Additionally, a feature extraction technique called Principal Component Analysis (PCA) was employed. According to PCA, 23 of the 30 characteristics account for 95% of the variation, indicating a considerable degree of repetition. To increase the feature space prior to feature selection, two engineering features, URL\_Risk\_Score and Domain\_Trust\_Score, were constructed using domain knowledge. Figure 9 highlights the top 15 characteristics with the highest Chi-Square rankings.

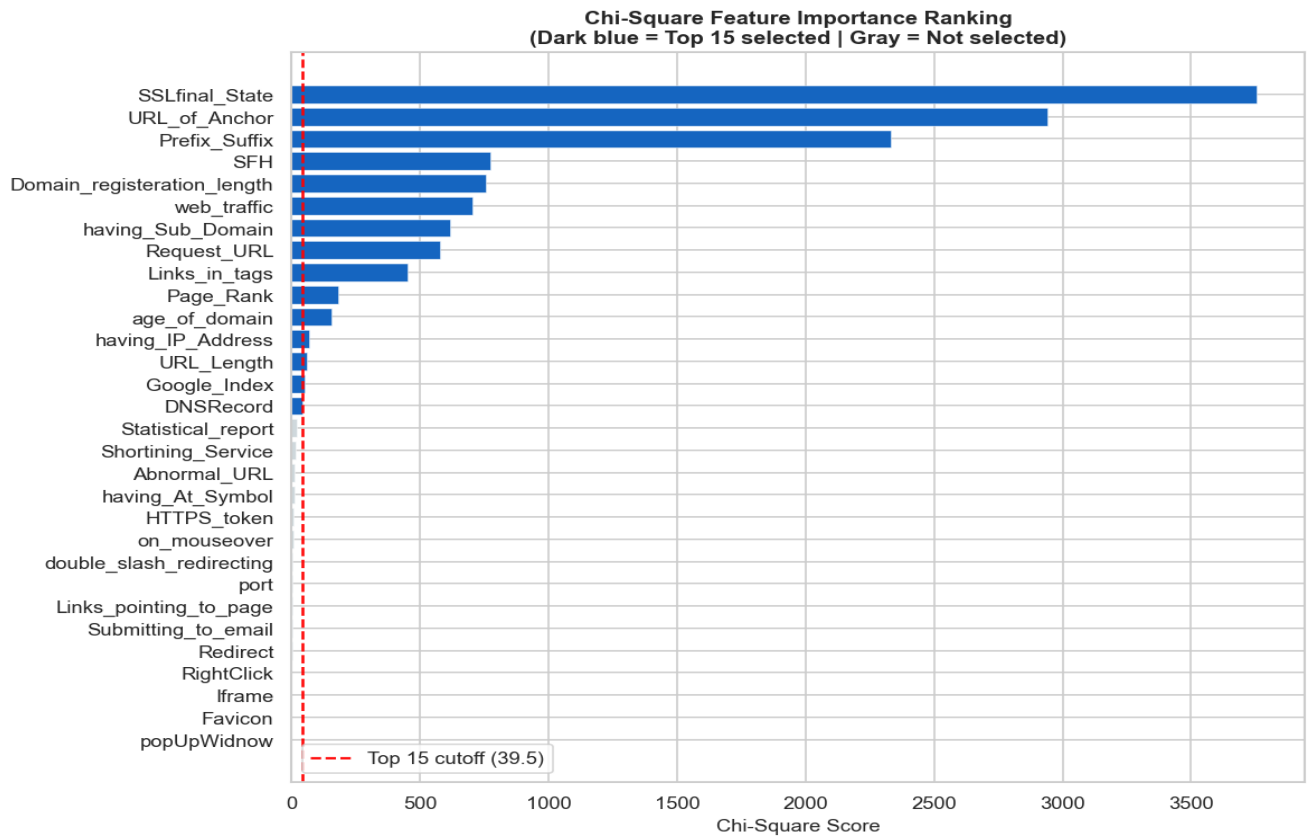


Figure 9 - Chi-Square feature importance bar chart

## 3.2 Supervised Learning

### 3.2.1 K-Nearest Neighbor (KNN)

The KNN is a non-parametric distance-based classification technique that uses the Euclidean distance to assign a new instance to the most common class label of its k closest neighbors in the training set (Cover & Hart, 1967). Because of its ease of use, interpretability, and demonstrated efficacy on numerical phishing datasets, KNN was selected as a baseline classifier (Chawla, 2022). By applying Standard Scaler to the features before training, the size of the features had no effect on the Euclidean distance computation. The model was tweaked to k=3, 5, 7, 9, and 11 after being trained using the full 30 feature set with k set to the default value of 5. The default KNN evaluation results are listed in Table 3.

| Metric          | Value  |
|-----------------|--------|
| Accuracy        | 94.89% |
| Precision       | 94.74% |
| Recall          | 93.67% |
| F1-Score        | 94.20% |
| False Negatives | 62     |

Table 3 - KNN Default Performance (k=5)

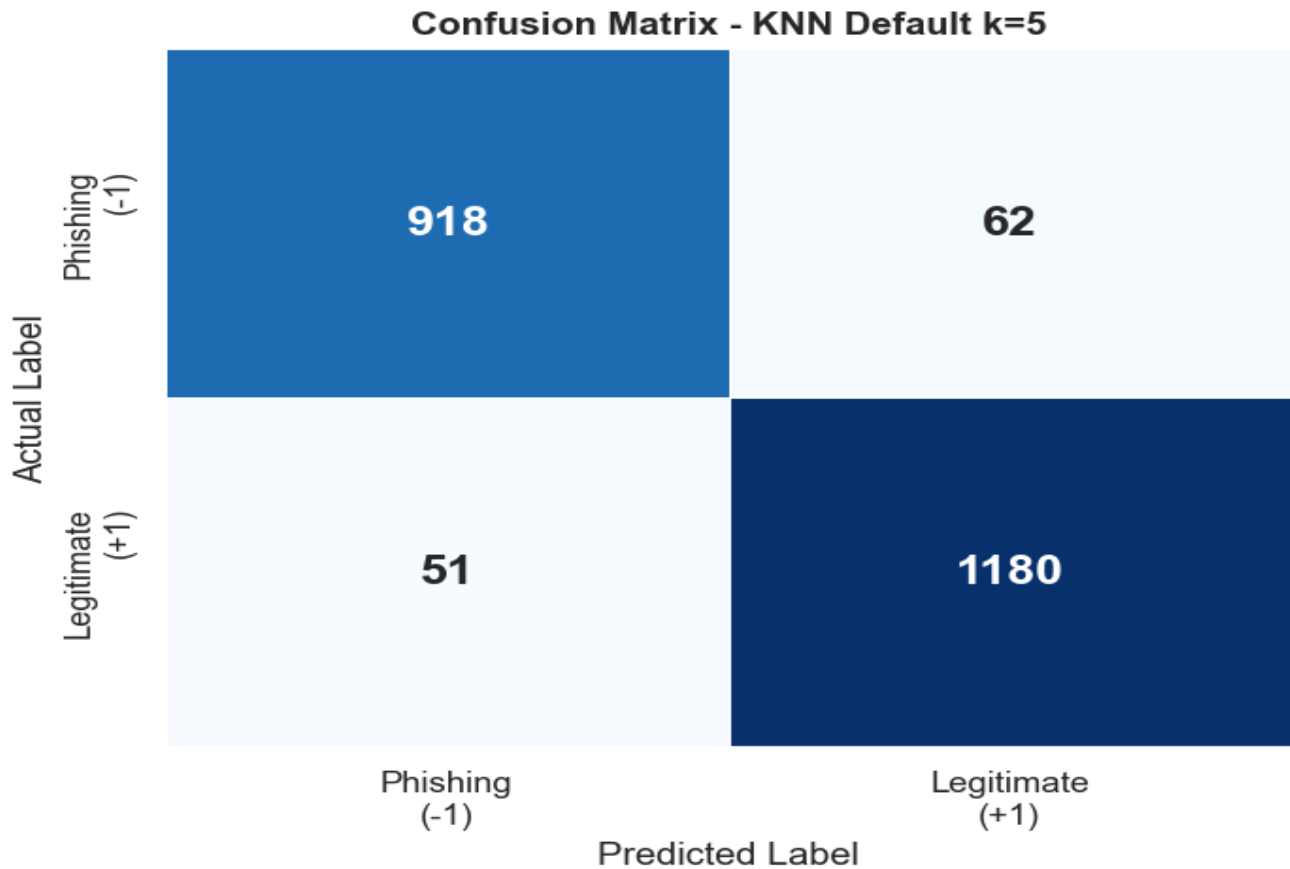


Figure 10 - KNN Confusion Matrix

### 3.2.2 Decision Tree

Decision Tree is a rule-based supervised classifier which recursively divides the feature space as a series of binary splits, generating human readable decision rules (Charbuty & Abdulazeez, 2021). It was chosen because of its interpretability, a property of vital importance in cybersecurity applications because, as well as the predictability of a classification decision, the reasoning behind that classification is often of critical importance (Tang & Mahmoud, 2021).

Decision Tree is scale-invariant and hence scaled data was not used for training. The split of SSLfinal\_State less than or equal to 0.5 at the top node separated 3,919 phishing items from 4,499 authentic ones at the root node, supporting the Chi-Square ranking result. Training the model was done with default parameters, and then the model was fine-tuned on the max\_depth of the tree to be 3, 5, 7 and None.

| Metric          | Value  |
|-----------------|--------|
| Accuracy        | 97.11% |
| Precision       | 97.22% |
| Recall          | 96.22% |
| F1-Score        | 96.72% |
| False Negatives | 37     |

Table 4 - Decision Tree Default Performance (depth=None)

### Confusion Matrix - Decision Tree Default

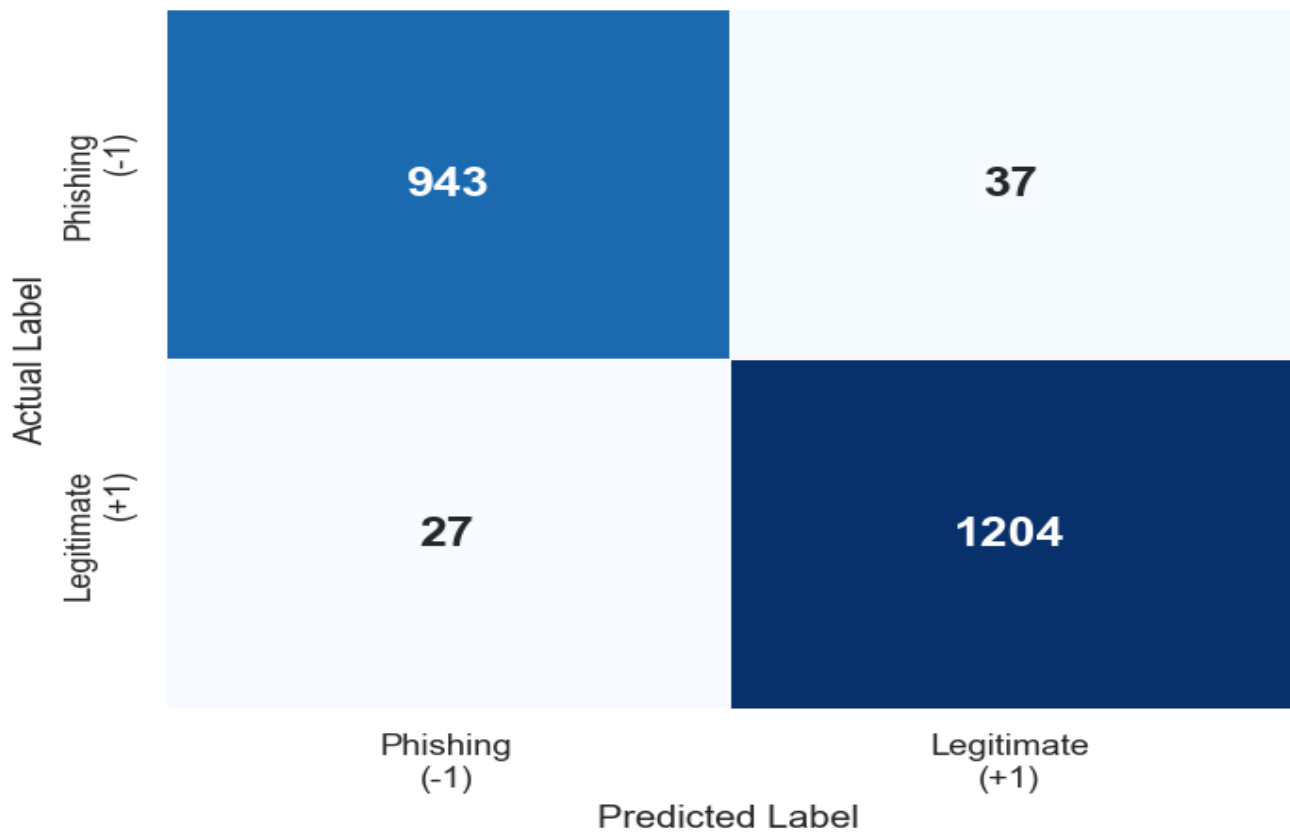


Figure 11 - Decision Tree Confusion Matrix

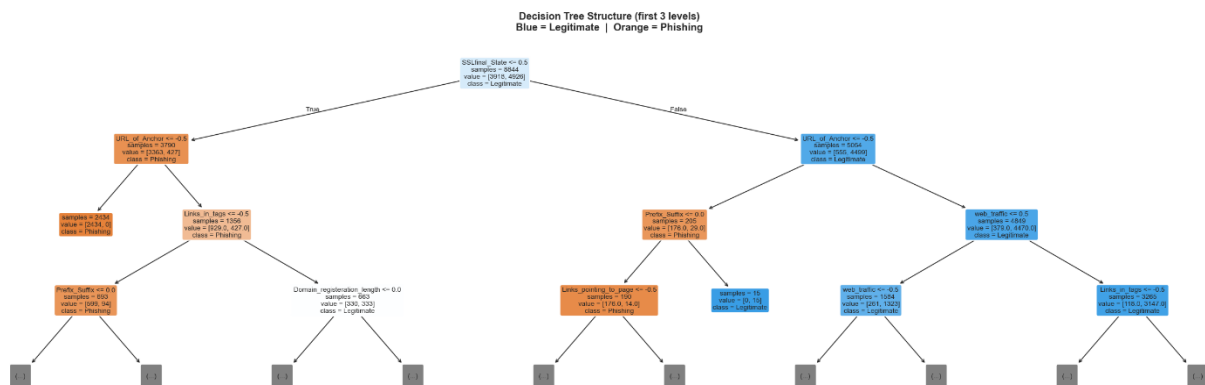


Figure 12 - Decision Tree Structure (first 3 levels)

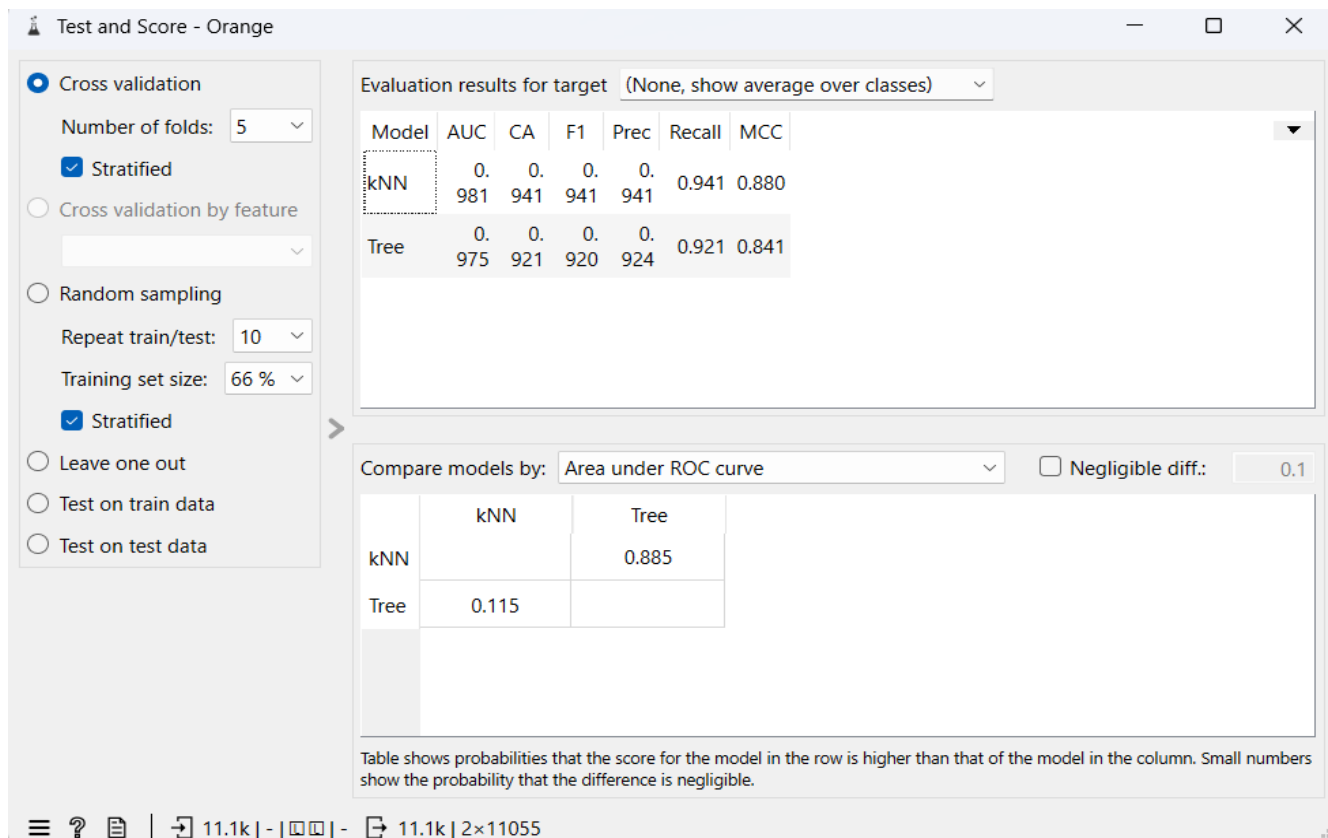


Figure 13 - Orange Test and Score results showing KNN performance

### 3.3 Unsupervised Learning - K-Means Clustering

K-Means is an unsupervised partitioning algorithm which partitions instances into  $k$  clusters to minimize the sum of squares of the within-cluster Euclidean distances (WCSS) (Ikotun et al., 2023). It was used to determine if there were any natural clusters in the data for the phishing and legitimate websites, without relying on labels, which is relevant for detecting zero-day phishing attacks that have not been labelled by any database (Hannousse & Yahiouche, 2021).

The entire data set was scaled before clustering using standard scaler. The ideal number of clusters was determined using the Elbow Method, and the outcome was verified using the Silhouette Score. Since the data set is binary classified, the ideal value of  $k$  in both situations was 2. The Elbow Method and Silhouette Score plots are shown in figure 14 and 15 respectively.

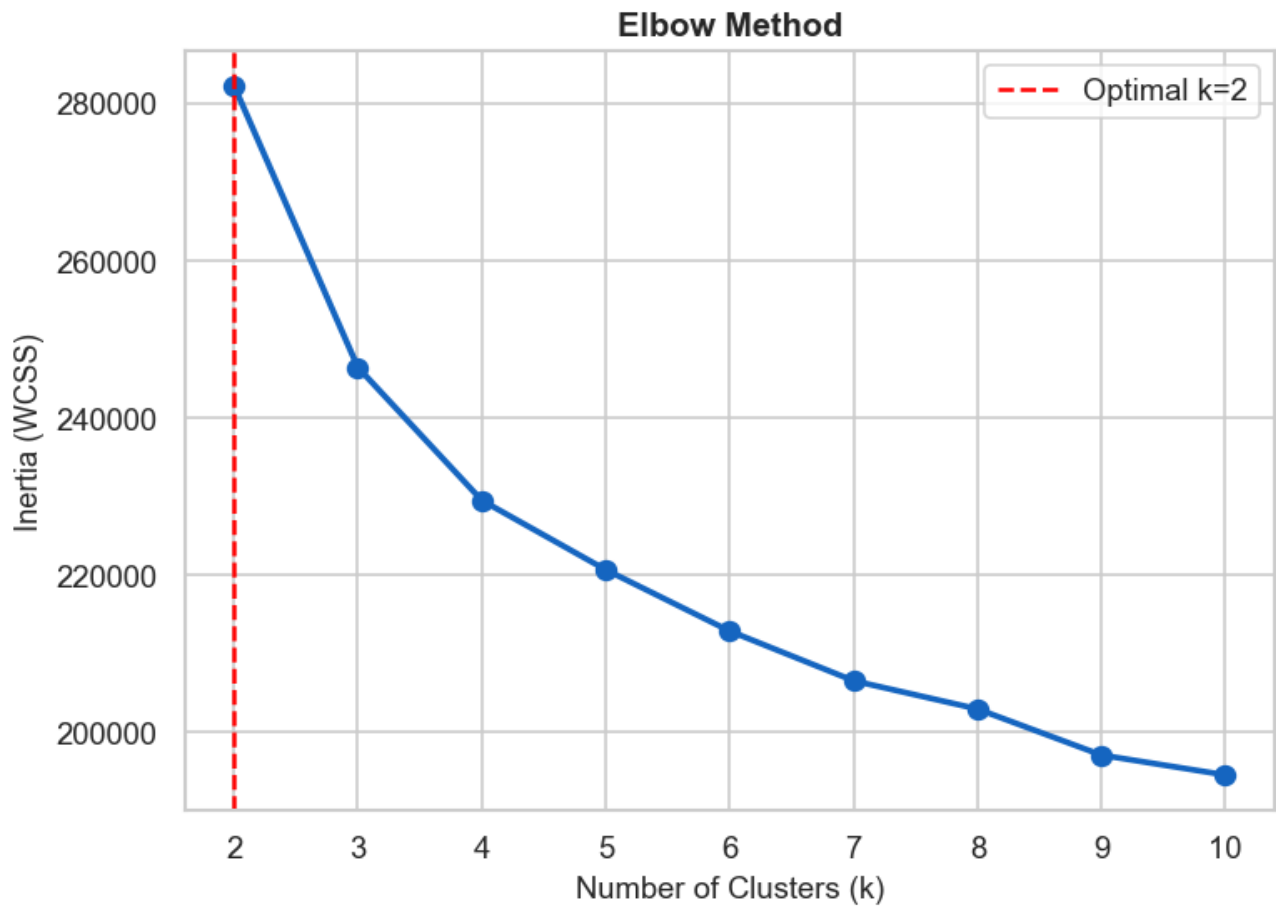


Figure 14 - Elbow Method plot



Figure 15 - Silhouette Score vs k plot



| Metric           | Value                                             |
|------------------|---------------------------------------------------|
| Optimal k        | 2                                                 |
| Silhouette Score | 0.2795                                            |
| Cluster 0 Purity | Legitimate: 56.2%, Phishing: 43.8%, Purity: 0.562 |
| Cluster 1 Purity | Legitimate: 53.4%, Phishing: 46.6%, Purity: 0.534 |

Table 5 - K-Means Clustering Results (k=2)

The score of Silhouette is 0.2795 which is moderate and exists in the range of weak cluster separation (0.25-0.5). This is normal, and it is clearly because of the type of feature encoding used. The Euclidean distance between instances is also highly restricted as all 30 features are restricted to three discrete values: -1, 0, and 1. However, many websites are equally close to both centroids, so K-Means fails to create tight and distinct boundaries. This is a recognized drawback of using distance-based clustering in the case of binary ordinal data (Hannousse & Yahiouche, 2021). 56.2% of legitimate websites were present in Cluster 0 and 53.4% of legitimate websites were present in Cluster 1 indicating that K-Means was unable to separate the two classes completely. This is not a problem with the algorithm and is a valid observation: phishing detection based on URL features alone, without the benefit of supervised learning (labels), is limited hence the need for the supervised learning approaches used in this project.

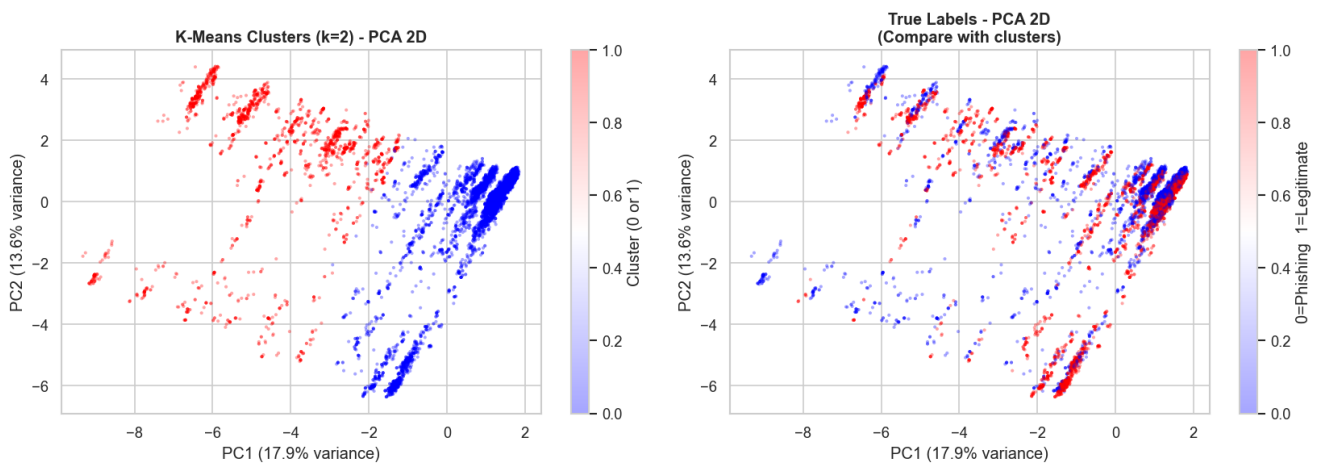


Figure 16 - PCA Scatter Plot (clusters vs true labels)

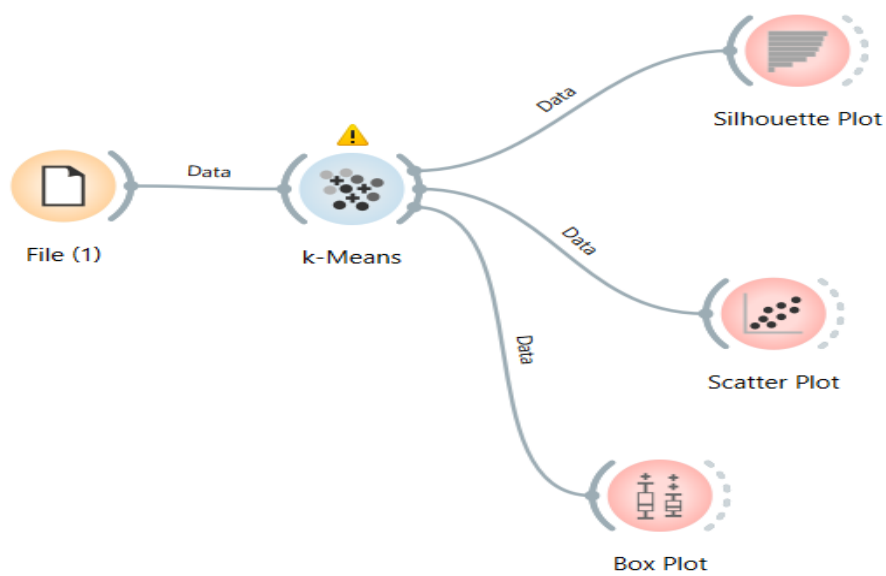


Figure 17 - Orange K-Means workflow

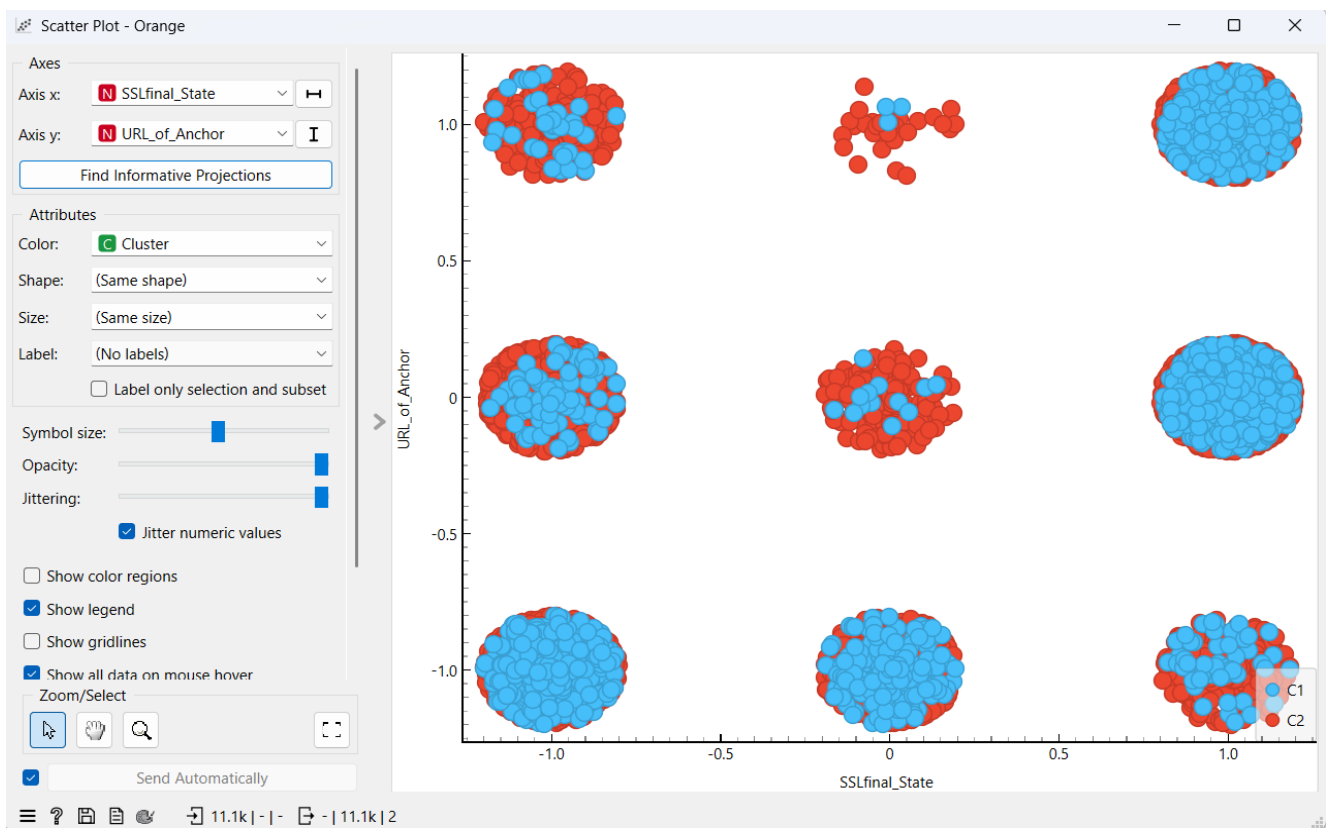


Figure 18 - Orange Scatter Plot (SSLfinal\_State vs URL\_of\_Anchor coloured by cluster)

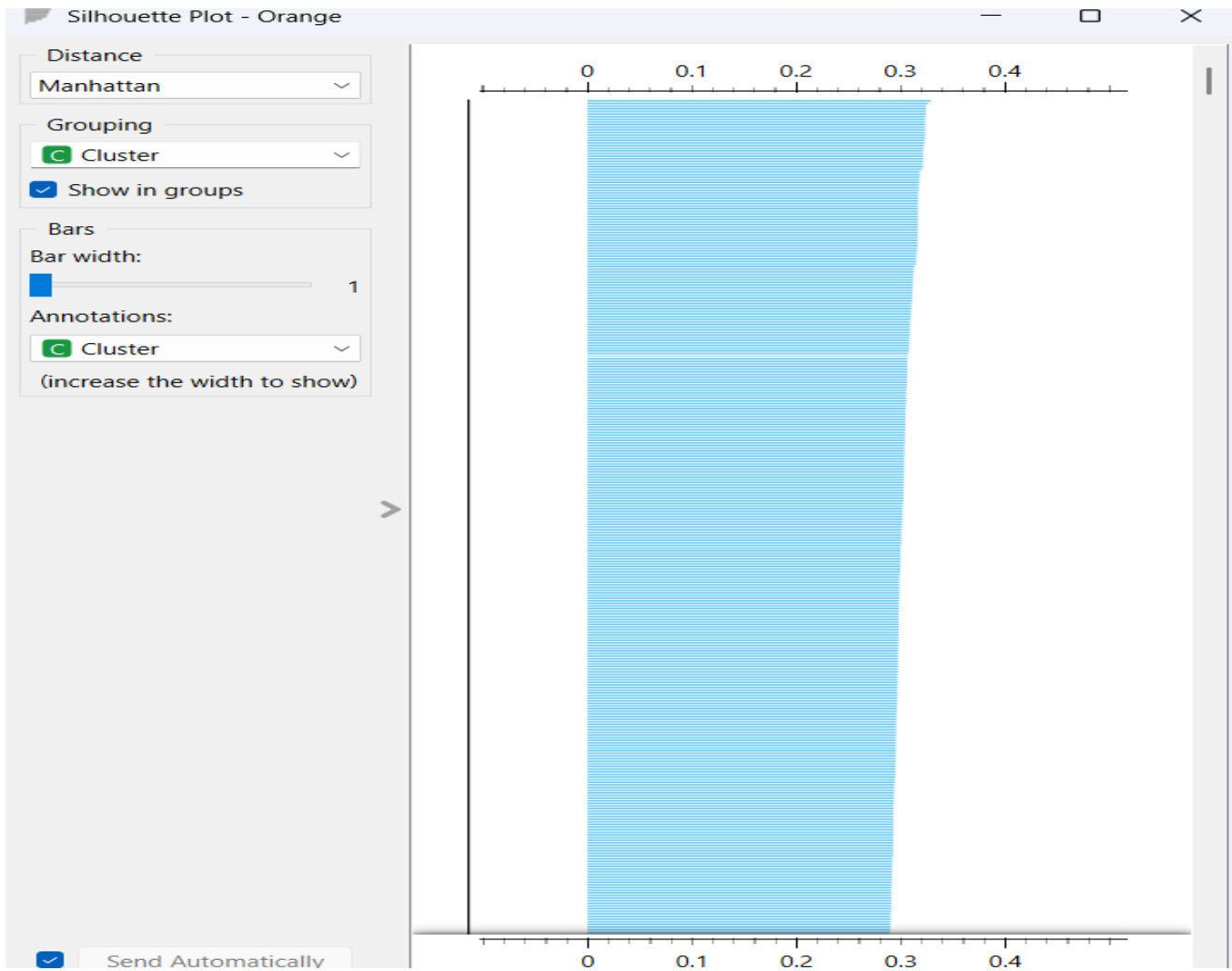


Figure 19 - Orange Silhouette Plot

### 3.4 Association Rule Mining - Apriori

Apriori association rule mining was utilized to identify the co-occurring URL-based phishing indicators in the phishing subset (4,898 occurrences) (Li et al., 2021). 128 frequent item sets were obtained by binarizing the features (valued -1, the phishing indicator was binary and valued as True) and setting a minimum support value of 0.3. The most intriguing finding was Prefix\_Suffix, which appeared on every phishing website in the sample, indicating that hyphens are a common feature of phishing. Phishing sites without DNS records consistently have a low Page rank, and the rule DNSRecord lead to Page\_Rank is deemed legitimate due to its high confidence level (92.8%) and high lift value (1.17). Table 6 displays the main association rules.

| Antecedent          | Consequent                     | Support | Confidence | Lift  |
|---------------------|--------------------------------|---------|------------|-------|
| Prefix_Suffix       | -                              | 100.0%  | -          | -     |
| Prefix_Suffix + SFH | -                              | 86.5%   | -          | -     |
| age_of_domain       | SSLfinal_State + Prefix_Suffix | 40.3%   | 75.0%      | 1.205 |
| DNSRecord           | Page_Rank                      | 32.5%   | 92.8%      | 1.170 |
| age_of_domain + SFH | URL_Length                     | 44.5%   | 95.2%      | 1.143 |

Table 6 - Top Association Rules from Apriori Mining

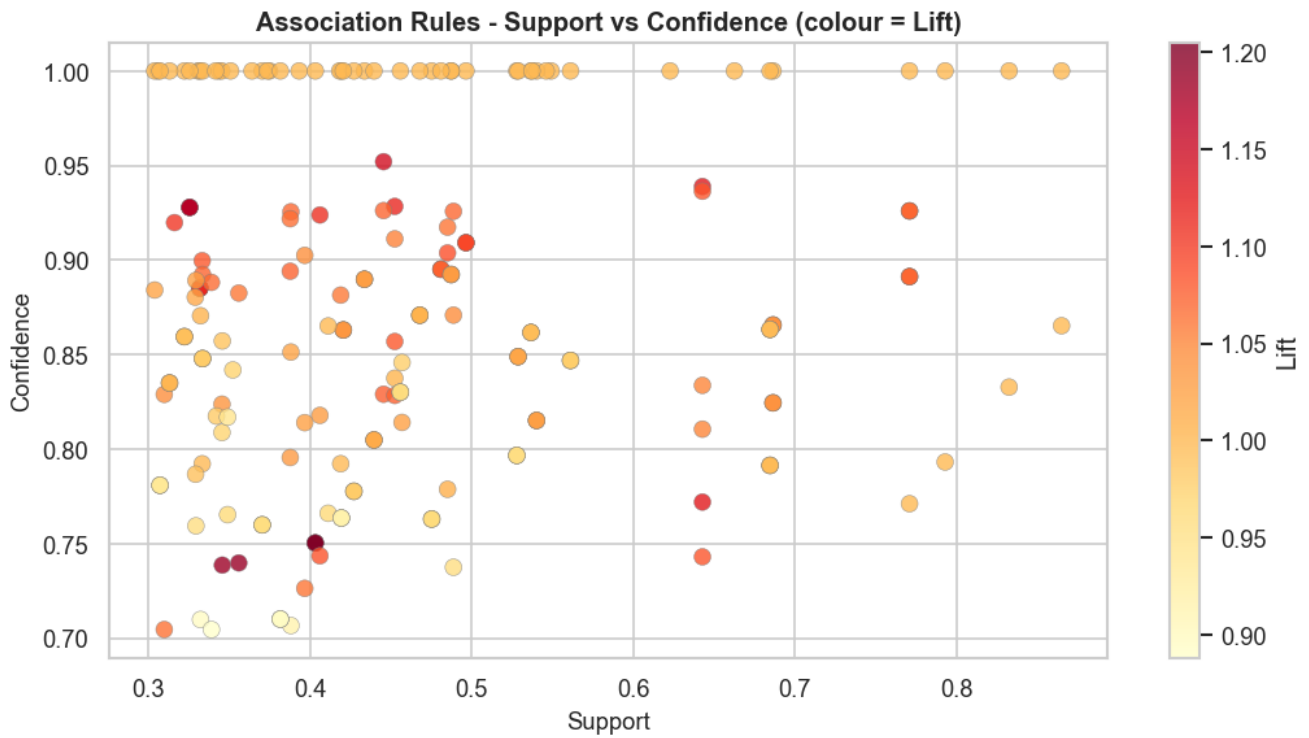


Figure 20 - Association Rules scatter plot (Support vs Confidence)

### 3.5 Evaluation Metrics

The three assessment measures listed below were applied to each of the three supervised models to compare them:

**Accuracy** is one way to conceptualize the total proportion of accurate classifications. As a broad rule of thumb, it is sufficient, but in unbalanced situations, it may be deceptive (Haq et al., 2024).

**Recall** (Sensitivity) is known as the percentage of real phishing sites that were accurately categorized. Our is the most crucial indicator for our project since a False Negative, or the assumption that a phishing site is legitimate, can result in consumers providing the attacker with their credentials, which is far more dangerous than a False Positive.

**F1-Score** is a harmonic average of precision and recall which gives a better balance of performance especially when there is a cost associated with both false positive and false negative (Chawla, 2022).

While the Cluster Purity was determined by comparing the cluster assignments with the actual class labels, the Silhouette Score was employed in the unsupervised K-Means technique to assess the degree of cohesiveness and separation of the clusters.

## Task 4: Hyper-parameter optimization – Summarization of results and inference

### 4.1 KNN Hyperparameter Tuning

The most important hyperparameter of KNN is the number of neighbors ( $k$ ). A small  $k$  is good at capturing the local patterns but has a risk of overfitting the noisy data; a large  $k$  is good at smoothing the decision boundary but may underfit the complex distribution (Cover & Hart, 1967). Five values were tested:  $k = 3, 5, 7, 9$ , and  $11$ . The results are shown in table 7.

| <b>k</b> | <b>Accuracy</b> | <b>Precision</b> | <b>Recall</b> | <b>F1-Score</b> |
|----------|-----------------|------------------|---------------|-----------------|
| <b>3</b> | <b>94.93%</b>   | <b>95.21%</b>    | <b>93.27%</b> | <b>94.23%</b>   |
| 5        | 94.89%          | 94.74%           | 93.67%        | 94.20%          |
| 7        | 94.53%          | 94.60%           | 92.96%        | 93.77%          |
| 9        | 94.39%          | 94.40%           | 92.86%        | 93.62%          |
| 11       | 94.08%          | 93.45%           | 93.16%        | 93.31%          |

Table 7 - KNN Hyperparameter Tuning Results

The performance of the program decreased steadily with the increase in  $k$  and as  $k$  became 3, the program got an F1-Score of 94.23%. The monotonic decrease indicates that the phishing and legitimate websites have cluster patterns in the scaled feature space, such that smaller neighborhoods of the scaled features space could define clearer classification boundaries.

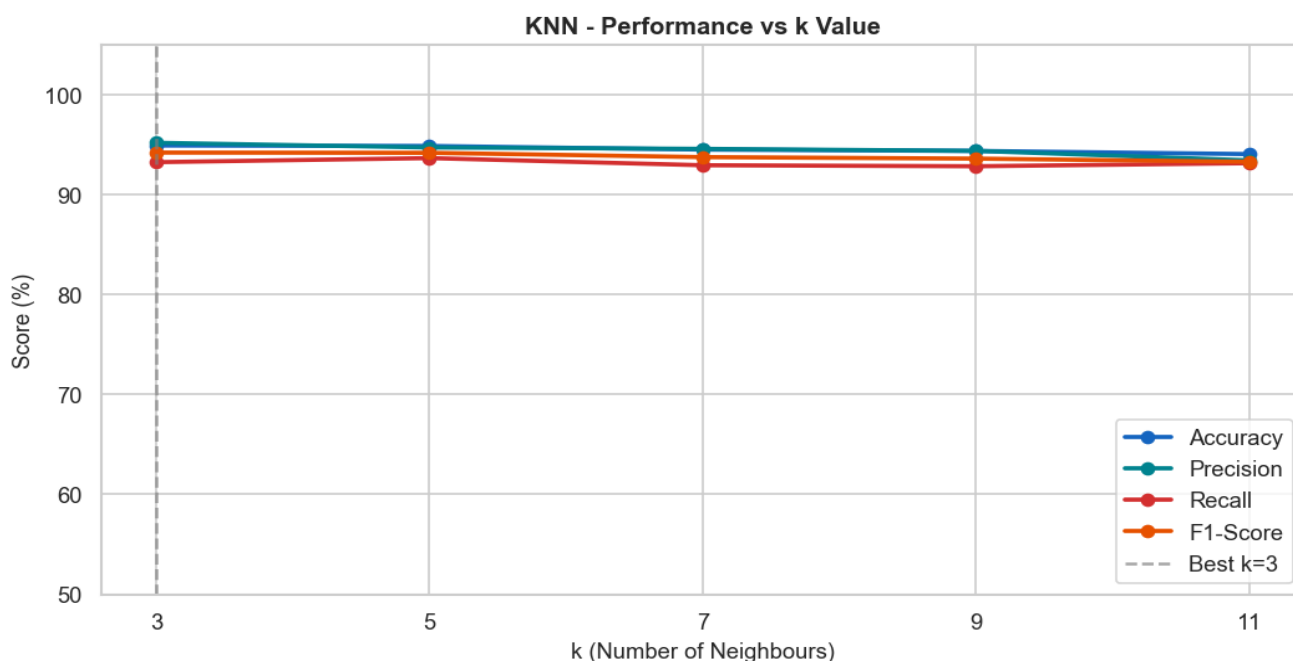


Figure 21- KNN Performance vs  $k$  (line chart)

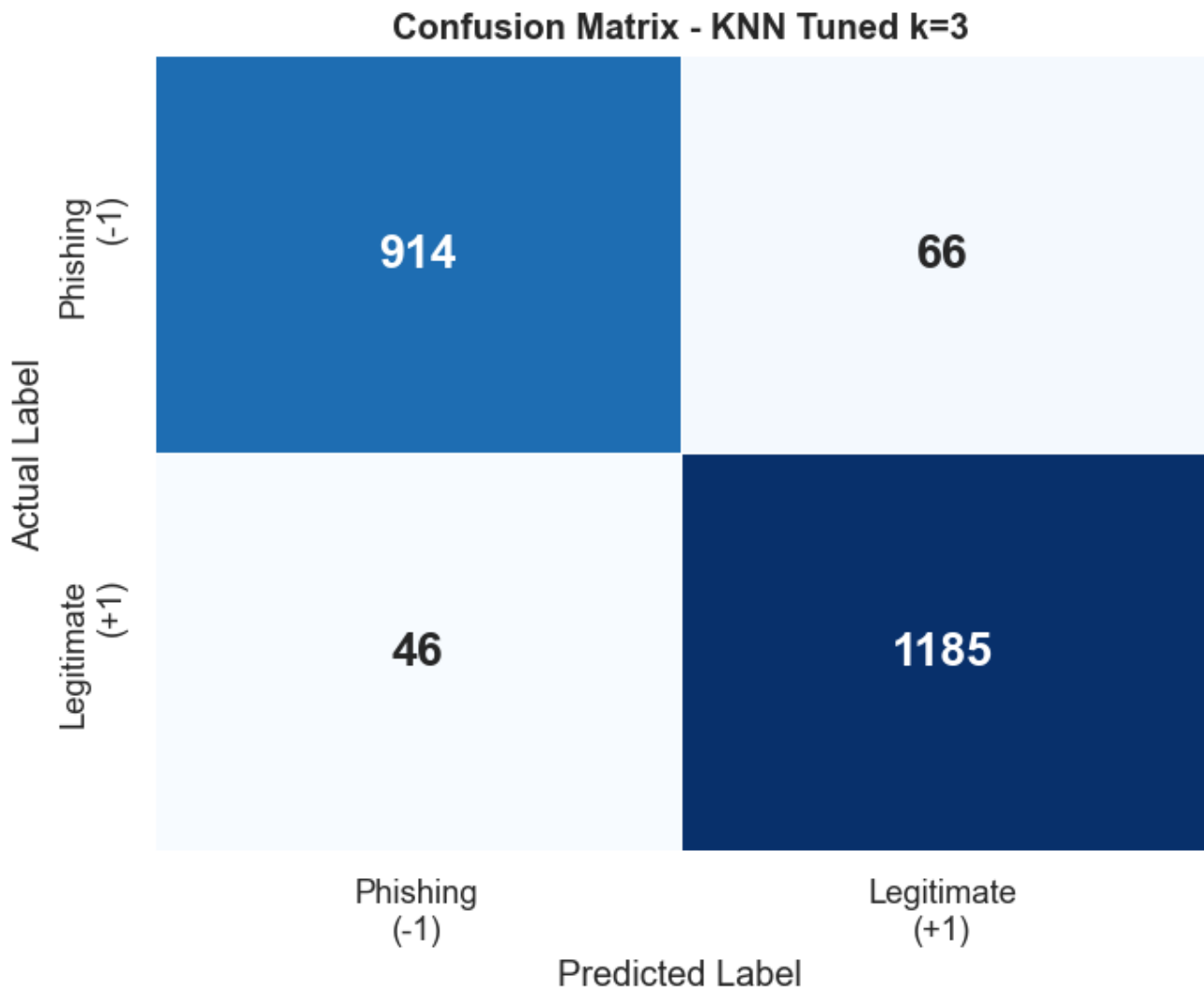


Figure 22 - KNN Tuned Confusion Matrix (k=3)

## 4.2 Decision Tree Hyperparameter Tuning

The maximum depth of the tree structure that will be constructed is denoted by `max_depth`. While limiting depth might help avoid overfitting, complicated boundaries in the data may lead to underfitting (Charbuty & Abdulazeez, 2021). Four distinct `max_depth` combinations were tested: 3, 5, 7, and None (no limit). The findings are shown in Table 8.

| max_depth   | Accuracy      | Precision     | Recall        | F1-Score      |
|-------------|---------------|---------------|---------------|---------------|
| 3           | 90.77%        | 96.97%        | 81.73%        | 88.70%        |
| 5           | 92.67%        | 94.85%        | 88.27%        | 91.44%        |
| 7           | 93.76%        | 93.76%        | 92.04%        | 92.89%        |
| None (full) | <b>97.11%</b> | <b>97.22%</b> | <b>96.22%</b> | <b>96.72%</b> |

Table 8 - Decision Tree Hyperparameter Tuning Results

Every metric was growing monotonically as the depth increased. On all metrics, the entire unconstrained tree outperformed the others, with recall at full depth reaching 96.22% as opposed to 81.73% at depth=3. Since the held-out test set continued to perform well, this suggests that there was no overfitting and that the boundaries in the dataset are genuinely complex and can only be accurately represented by deep rules. The total performance trend for each depth value is displayed in the figure below.

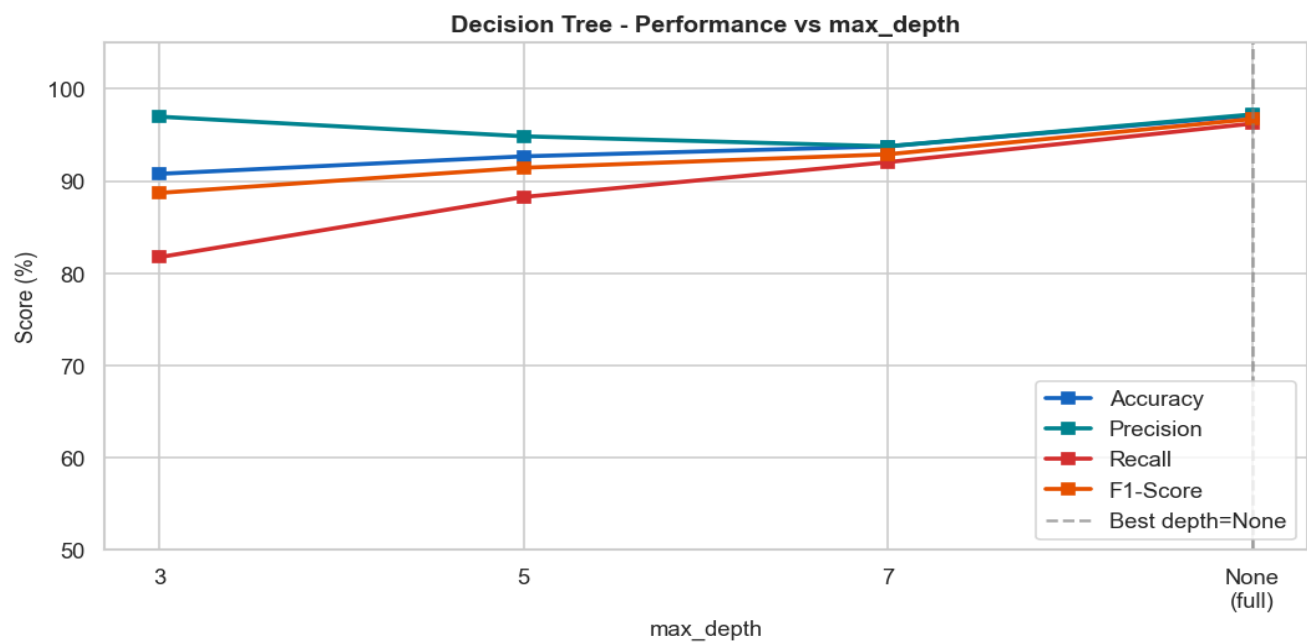


Figure 23 - Decision Tree Performance vs max\_depth (line chart)

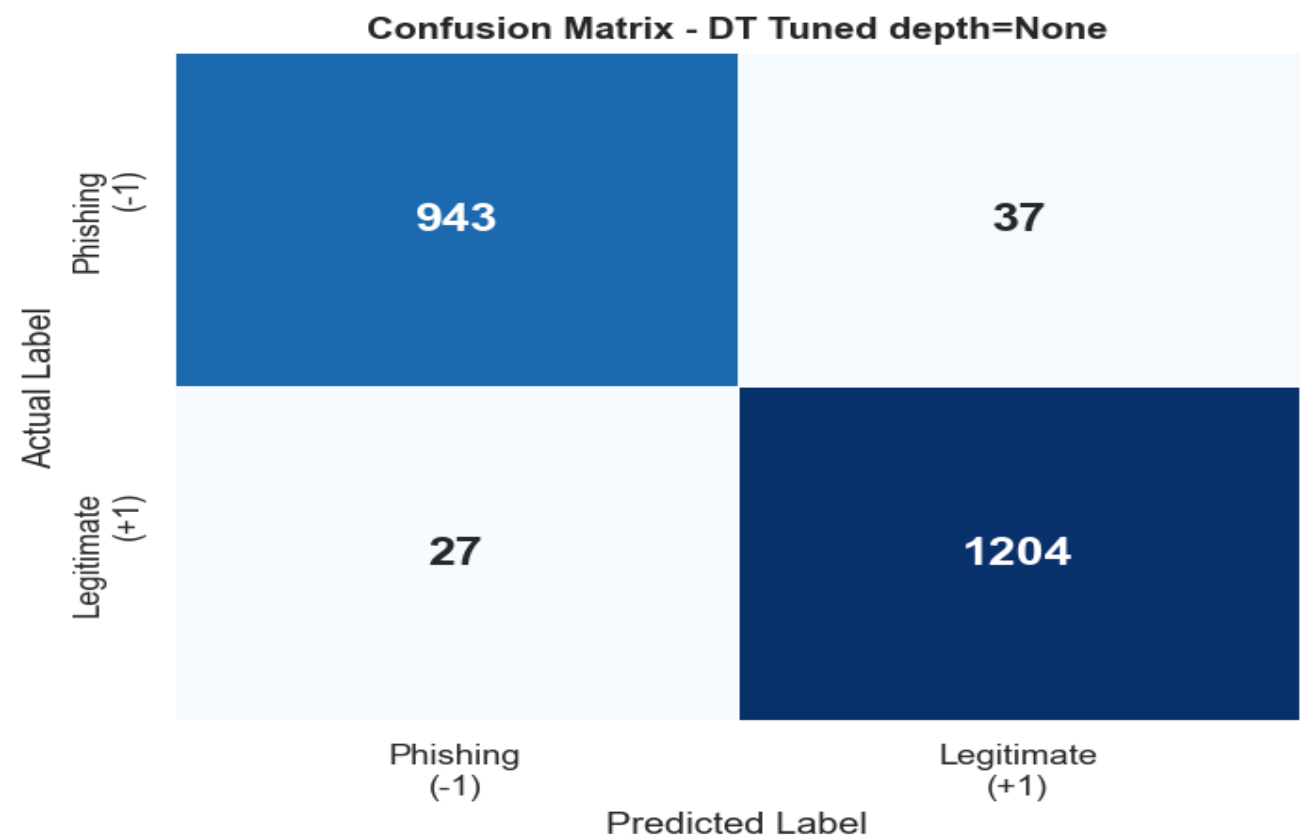


Figure 24 - Decision Tree Tuned Confusion Matrix (depth=None)

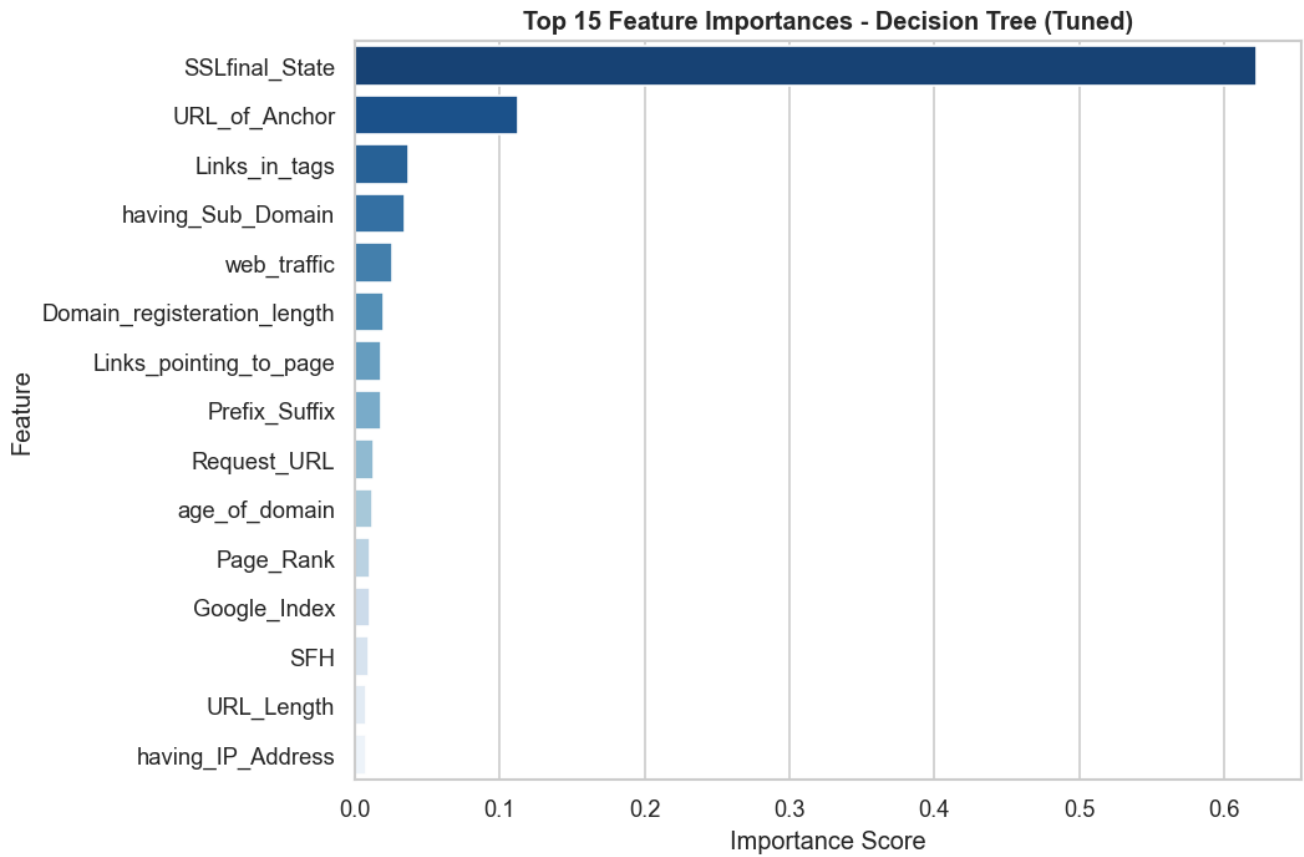


Figure 25 - Decision Tree Feature Importance chart

### 4.3 K-Means Hyperparameter Tuning

The ideal number of clusters (k), which ranged from two to ten, was chosen using the Elbow Method and the Silhouette Score as complimentary methods. Table 9 presents the main findings.

| k | Inertia (WCSS) | Silhouette Score |
|---|----------------|------------------|
| 2 | 282,208.8      | 0.2760           |
| 3 | 246,378.7      | <b>0.2811</b>    |
| 4 | 229,464.2      | 0.1105           |
| 5 | 220,639.0      | 0.1095           |

Table 9 - K-Means Elbow Method and Silhouette Scores

The Silhouette Score is slightly higher at k=3 with a value of 0.2811 compared to 0.2795 at k=2, but k=2 was chosen as the optimal k value. There are only two meaningful classes in the dataset (phishing and legitimate) and adding a third cluster would form an artificial group that would have no semantic or practical relevance. Moreover, the Silhouette Score is significantly lower at k=4, which again indicates the presence of only two significant natural clusters in the data.



## 4.4 MindSpore MLP Hyperparameter Tuning

The MindSpore MLP was tuned under six experiments of epoch size, learning rate, and batch size following the lab manual pipeline (Huawei, 2023). Table 10 presents the findings.

| Epochs     | Learning Rate | Batch Size | Accuracy      |
|------------|---------------|------------|---------------|
| 20         | 0.001         | 64         | 95.57%        |
| 20         | 0.001         | 32         | 95.84%        |
| 50         | 0.001         | 64         | 96.20%        |
| <b>100</b> | <b>0.001</b>  | <b>64</b>  | <b>96.47%</b> |
| 50         | 0.005         | 64         | 46.38%        |
| 100        | 0.005         | 32         | 46.38%        |

Table 10 - MindSpore MLP Hyperparameter Tuning Results

The lowest number of FN was 26 with an accuracy of 96.47% when the ideal epoch, lr, and batch were 100, 0.001, and 64, respectively. The steady learning rate for this binary ordinal dataset is  $lr=0.001$ , as confirmed by configurations with  $lr=0.005$  collapsing to a 46.38% accuracy, which is equivalent to guessing for a binary issue. Figure 26 displays accuracy for every setting.

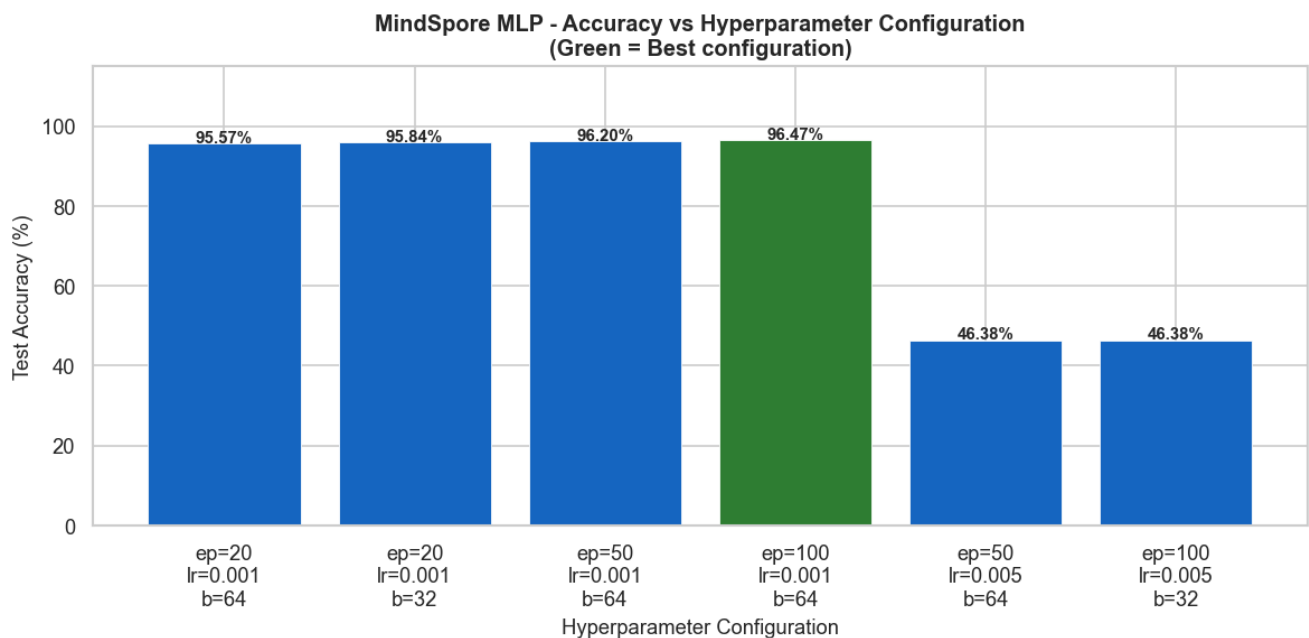


Figure 26 - MindSpore Hyperparameter Accuracy bar chart

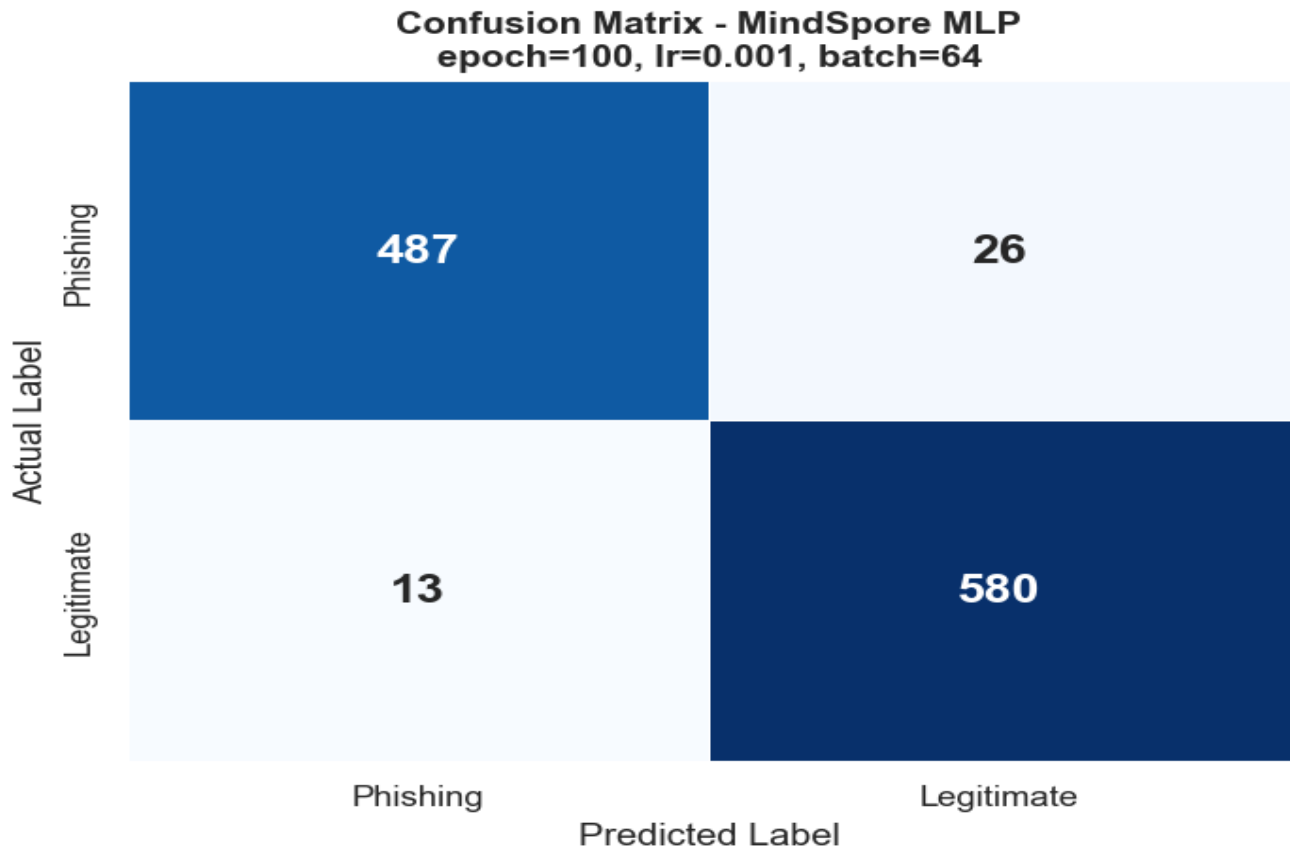


Figure 27 - MindSpore MLP Confusion Matrix

## 4.5 Summary of All Models

Table 11 shows the consolidated performance of all the models after tuning and the consolidated basis is used for comparison and inference.

| Model                      | Framework                | Accuracy      | Precision     | Recall           | F1-Score      | False Negatives |
|----------------------------|--------------------------|---------------|---------------|------------------|---------------|-----------------|
| KNN (k=5) default          | scikit-learn             | 94.89%        | 94.74%        | 93.67%           | 94.20%        | 62              |
| KNN (k=3) tuned            | scikit-learn             | 94.93%        | 95.21%        | 93.27%           | 94.23%        | 66              |
| Decision Tree default      | scikit-learn             | 97.11%        | 97.22%        | 96.22%           | 96.72%        | 37              |
| Decision Tree tuned        | scikit-learn             | 97.11%        | 97.22%        | 96.22%           | 96.72%        | 37              |
| MindSpore MLP default      | MindSpore                | 96.02%        | —             | —                | —             | —               |
| <b>MindSpore MLP tuned</b> | <b>MindSpore</b>         | <b>96.47%</b> | <b>97.40%</b> | <b>94.93%</b>    | <b>96.15%</b> | <b>26</b>       |
| K-Means (k=2)              | scikit-learn /<br>Orange | —             | —             | Silhouette: 0.28 | —             | —               |

Table 11 - Consolidated Model Performance Summary

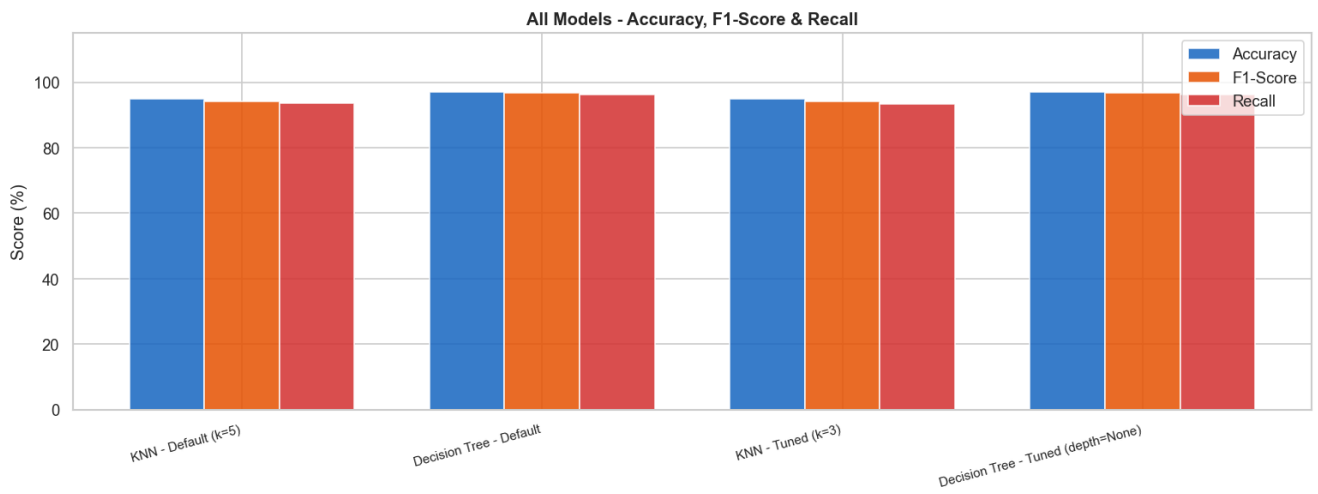


Figure 28 - All Models Comparison bar chart (Accuracy, F1, Recall)

## Task 5: Discussions and Future Extensions to the Work

### 5.1 Critical Analysis of Results

With the Decision Tree Classifier displaying the highest accuracy of 97.11% and recall of 96.22%, the results of this experimental study demonstrate that the supervised learning approach is significantly better than the unsupervised learning approach in the detection of phishing websites on the UCI dataset.

Because the splitting criterion is based on rules that are naturally adapted to the discrete ordinal character of the data, the Decision Tree performs better. The root node split on `SSLfinal_State < 0.5` alone was sufficient to accurately divide most of the 8,844 training cases; this split was continuously selected as the most discriminative feature (Chi-Square ranking  $\chi^2=3,753.8$ , Pearson correlation  $r=0.71$ , and feature significance 0.622). This triangulation of three independent analytical methods provides compelling evidence that the primary determinant of phishing intent is the authenticity of the SSL certificate (Hannousse & Yahiouche, 2021).

In every metric, KNN fared worse than the Decision Tree, despite being stable and dependable. By limiting the Euclidean distance variation in the feature space, the binary ordinal encoding (-1, 0, 1) reduces the discriminative ability of proximity-based classification. This is consistent with the findings of Haq et al. (2024), who reported that distance classifiers are particularly sensitive to feature representation quality.

Despite having a lower overall accuracy (96.47%) than Decision Tree, the MindSpore MLP has the fewest false negatives of all the models, just 26 as opposed to Decision Tree's 37 and KNN default's 62. In the context of cybersecurity, a false negative is an assumption that a phishing site is secure, which puts consumers in danger of losing their login credentials. Thus, the most useful deployed model in this research is MindSpore MLP. Phishing and genuine websites are not organically grouped in the binary feature space, according to the K-Means algorithm findings, which yielded a Silhouette Score of 0.2795. The result is consistent with the broader literature that supervised techniques must be employed for phishing detection as unsupervised techniques alone are insufficient (Tang & Mahmoud, 2021).

The findings of Apriori association rule mining revealed that `Prefix_Suffix` is a deterministic phishing signature that can be utilized as a high confidence rule in a production system and is present on all phishing websites (100%).

## 5.2 Comparison to Previous Work

Table 12 presents the results of this study in the context of previous research on the Phishing Dataset.

| Study                        | Algorithms                      | Best Accuracy             | Feature Selection       | Unsupervised         | No-code Tool        |
|------------------------------|---------------------------------|---------------------------|-------------------------|----------------------|---------------------|
| Chawla (2022)                | RF + DT + KNN ensemble          | 97.73%                    | No                      | No                   | No                  |
| Choudhary et al. (2023)      | RF + feature selection          | 97.87% (UCI)              | Yes (k-fold)            | No                   | No                  |
| Haq et al. (2024)            | 8 algorithms incl. CNN          | 98%+ (with DL)            | Yes                     | No                   | No                  |
| Hannousse & Yahiouche (2021) | Multiple classifiers            | Benchmark study           | Partial                 | No                   | No                  |
| <b>This Project</b>          | <b>KNN + DT + K-Means + MLP</b> | <b>97.11% (single DT)</b> | <b>Yes (Chi-Square)</b> | <b>Yes (K-Means)</b> | <b>Yes (Orange)</b> |

Table 12 - Comparison with Related Work

With four additional analytical dimensions unsupervised clustering, Chi-Square feature selection, association rule mining, and migration from the old to the new framework, MindSpore the project's single Decision Tree produced 97.11%, which was comparable to Chawla's (2022) ensemble of three models and 97.73%. None of the research referenced above includes the MindSpore MLP, which had the fewest false negatives (26).

## 5.3 Pros and Cons of Each Model

| Model                | Pros                                              | Cons                                                               |
|----------------------|---------------------------------------------------|--------------------------------------------------------------------|
| <b>KNN</b>           | Simple, no training phase, interpretable          | Sensitive to feature scale, slower on large datasets, lower recall |
| <b>Decision Tree</b> | Interpretable, scale-invariant, fast, high recall | May overfit without depth restriction on noisy data                |
| <b>K-Means</b>       | No labels required, discovers hidden structure    | Cannot cleanly separate binary ordinal features, modest Silhouette |
| <b>MindSpore MLP</b> | Fewest false negatives, scalable, deep learning   | Requires careful lr tuning, less interpretable than DT             |

Table 13 - Model Pros and Cons

## 5.4 How the Framework Addresses the Objectives

Every one of the five project goals listed in Task 1 was accomplished:

- KNN and Decision Tree were implemented, tuned, and evaluated. The tuned Decision Tree (depth=None) achieved the highest accuracy of 97.11% and recall of 96.22% with only 37 false negatives
- K-Means clustering (k=2) was employed, and the Elbow Method and Silhouette Score were utilized for validation.
- Chi-Square feature selection was used to minimize the feature space from 30 to 15 key features.
- After each of the three algorithms' hyperparameters were adjusted, comparative tables were created.
- The complete pipeline was successfully migrated to MindSpore, achieving a best accuracy of 96.47% with only 26 false negatives, the fewest of any model tested.

## 5.5 Limitations and Future Work

### 5.5.1 Limitations

Three limitations are acknowledged in this study as shown below:

- Despite the dataset's 2012 origins, several papers published between 2021 and 2024 attest to their ongoing significance as a standard in phishing detection research (Chawla, 2022; Hannousse & Yahiouche, 2021).
- For both clustering and distance-based approaches, the binary ordinal encoding (-1, 0, 1) lowers the granularity. When features have just three discrete values, KNN performs poorly because many occurrences are equally spaced and neighborhood boundaries become less precise, even if it performs well with Euclidean distance. The inability to effectively distinguish the cluster centroids in a binary feature space is also the source of the very low K-Means Silhouette Score of just 0.2795.
- The MindSpore MLP training showed instability under all configurations at a higher learning rate (lr=0.005). Therefore, the loss function converged at 46.38% accuracy, which is equivalent to random prediction for a binary classification issue. To achieve more steady convergence in subsequent tests, more systematic learning rate scheduling or adaptive optimizers may be required due to the binary-ordinal structure and extremely high sensitivity to the optimizer's step size.

### 5.5.2 Future Work

There are three proposed extensions. First, the models' generalizability outside the UCI benchmark will be tested using this framework on the 2021 Mendeley Phishing Dataset (87 continuous features, 50/50 class distribution). Second, Choudhary et al. (2023) and Haq et al. (2024) revealed that ensemble techniques such as Random Forest or XGBoost may be directly compared to their best-performing configurations. Third, the migration work completed in this project makes sense if they implement the MindSpore MLP as a real-time browser plugin with the MindSpore ModelArts cloud platform.

## Use of GenAI

Claude (Anthropic) was used as a supporting tool in this project. Claude was specifically used to help and debug parts of the Python and MindSpore code in the Jupyter notebook, seek and summarize pertinent peer-reviewed research publications. It was used also to create APA style citations. It was also used to support the structuring and drafting of sections of this report.

## References

- Charbuty, B., & Abdulazeez, A. (2021). Classification based on decision tree algorithm for machine learning. *Journal of Applied Science and Technology Trends*, 2(01), 20–28. <https://doi.org/10.38094/jastt20165>
- Chawla, A. (2022). Phishing website analysis and detection using machine learning. *International Journal of Intelligent Systems and Applications in Engineering*, 10(1), 10–16. <https://ijisae.org/index.php/IJISAE/article/view/1333/679>
- Choudhary, T., Mhapankar, S., Bhddha, R., Kharuk, A., & Patil, R. (2023). A machine learning approach for phishing attack detection. *Journal of Artificial Intelligence and Technology*, 3(3), 108–113. [A Machine Learning Approach for Phishing Attack Detection | Journal of Artificial Intelligence and Technology](#)
- Cover, T., & Hart, P. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1), 21–27. [Nearest neighbor pattern classification | IEEE Journals & Magazine | IEEE Xplore](#)
- Demšar, J., Curk, T., Erjavec, A., Gorup, Č., Hočevár, T., Milutinovič, M., Možina, M., Polajnar, M., Toplak, M., Starič, A., Štajdohar, M., Umek, L., Žagar, L., Žbontar, J., Žitnik, M., & Zupan, B. (2013). Orange: Data mining toolbox in Python. *Journal of Machine Learning Research*, 14, 2349–2353.
- FBI. (2023). Internet crime report 2023. Federal Bureau of Investigation. [https://www.ic3.gov/Media/PDF/AnnualReport/2023\\_IC3Report.pdf](https://www.ic3.gov/Media/PDF/AnnualReport/2023_IC3Report.pdf)
- Hannousse, A., & Yahiouche, S. (2021). Towards benchmark datasets for machine learning based website phishing detection: An experimental study. *Engineering Applications of Artificial Intelligence*, 104, 104347. <https://doi.org/10.1016/j.engappai.2021.104347>
- Haq, I., Khan, M. A., & Alqahtani, A. (2024). Comparative evaluation of machine learning algorithms for phishing site detection. *PLOS ONE*. <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC11232597/>
- Huawei Technologies. (2023). MindSpore documentation: MindSpore framework for AI development. <https://www.mindspore.cn/en>
- Ikotun, A. M., Ezugwu, A. E., Abualigah, L., Abuhaija, B., & Heming, J. (2023). K-means clustering algorithms: A comprehensive review, variants analysis, and advances in the era of big data. *Information Sciences*, 622, 178–210. <https://doi.org/10.1016/j.ins.2022.11.139>
- Li, Z., Li, X., Tang, R., & Zhang, L. (2021). Apriori algorithm for the data mining of global cyberspace security issues for human participatory based on association rules. *Frontiers in Psychology*, 11, 582480. <https://doi.org/10.3389/fpsyg.2020.582480>
- Mohammad, R., & McCluskey, L. (2012). Phishing websites [Dataset]. UCI Machine Learning Repository. <https://doi.org/10.24432/C51W2X>



Tang, L., & Mahmoud, Q. H. (2021). A survey of machine learning-based solutions for phishing website detection. *Machine Learning and Knowledge Extraction*, 3(3), 672–694. <https://doi.org/10.3390/make3030034>